

Database & Storage Deep Dive

How databases really store, scale, shard and stay reliable under load.

7 chapters · compiled from the Evolving Engineer — pradyumnachippigiri.substack.com

By Pradyumna Chippigiri

Compiled 4 October 2026

WHAT'S INSIDE

- How databases store data on the disk
- Scaling the Databases: Choosing the right strategy
- Nail Sharding in System Design Interviews: Range-Based, Hash-Based, Directory-Based
- SQL vs NoSQL: A Simple Checklist to Choose the Right Database in Interviews
- Relational Databases & ACID Transactions
- Relational Database & ACID Transactions : Part 2
- How does the database guarantee reliability using write-ahead logging?

CONTENTS

How databases store data on the disk

2026-02-19 · Easy explanation of how HDDs and SSDs store data from database

Source: <https://pradyumnachippigiri.substack.com/p/how-databases-store-data-on-the-disk>

Appreciate you reading this newsletter, I am writing as I learn and explore these topics in depth. I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my personal website - <https://pradyumnachippigiri.dev/blogs/database-storage-hdd-ssd..>

Also, checkout all my Linkedin tech posts here -> <https://pradyumnachippigiri.dev/blogs/linkedin-posts>

Here's what I learned this week

Unique features of OrderedDict in Python

RLE Run Length Encoding

Best resource for understanding OAuth

Let Start,,

Understanding how databases store table data such as rows and columns on disk forms the foundational groundwork for understanding how databases work internally. It helps us understand how indexes operate, how multi-level indexes are structured, and how B-trees and B+ trees efficiently store and access indexed data. But before that lets understand this from ground up

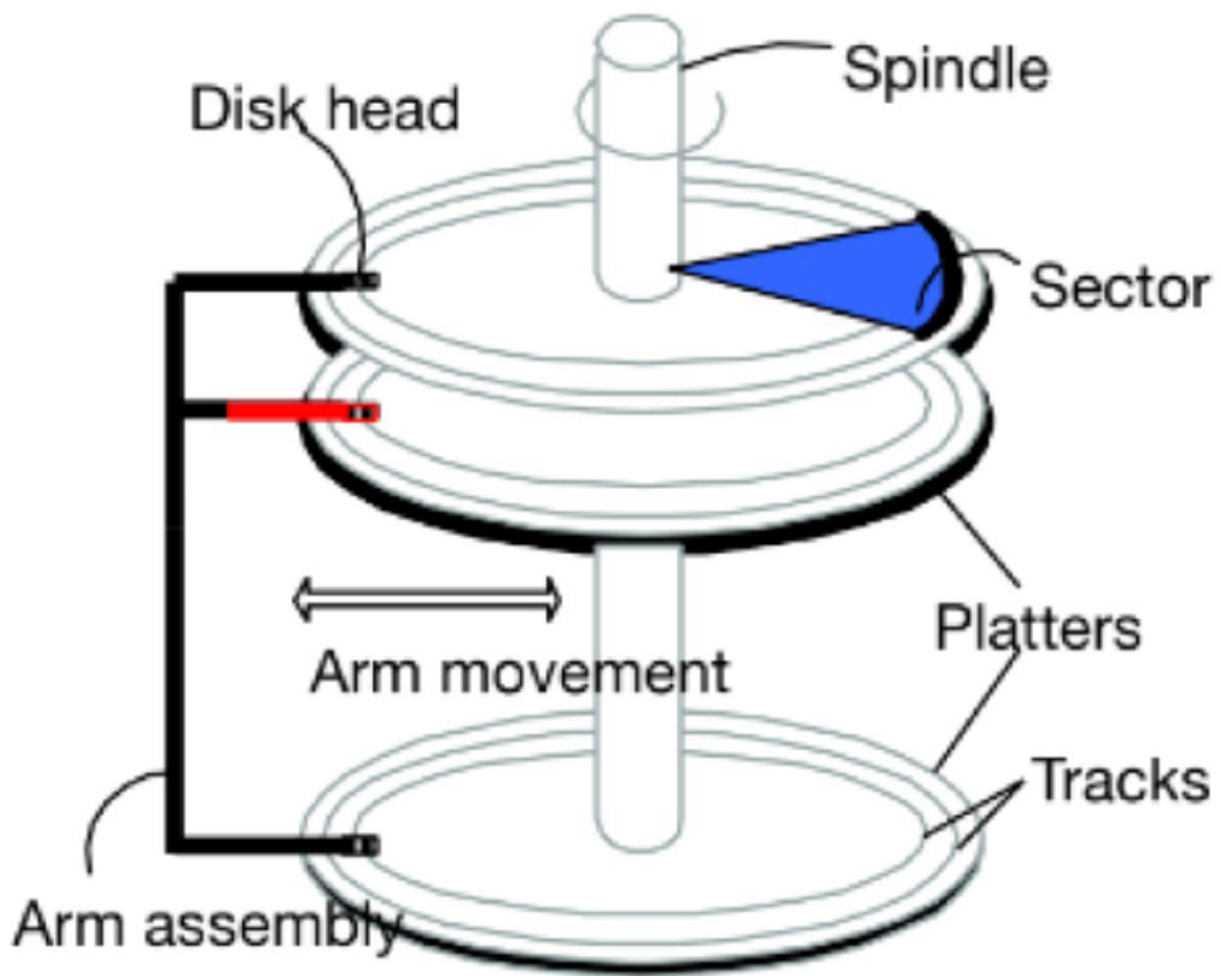
Let's see how it works below for HDDs, and then the similar concept applies to the SSDs.

Disk and Disk Structure

A disk is a non-volatile storage device used to store data persistently. Because it is non-volatile, the data on disk remains intact even when the system loses power (unlike RAM). This is why disks are essential for storing permanent files, databases, and operating system data.

Would highly recommend you watch this 3-minute youtube video, to get an idea of how an actual real HDD looks, so that you get a sense of what we are going to discuss in this article today :

The disk is composed of the following components :



Platters: Magnetic disks where data is recorded on both surfaces; and each surface is nothing but a set of concentric circles. (logic)

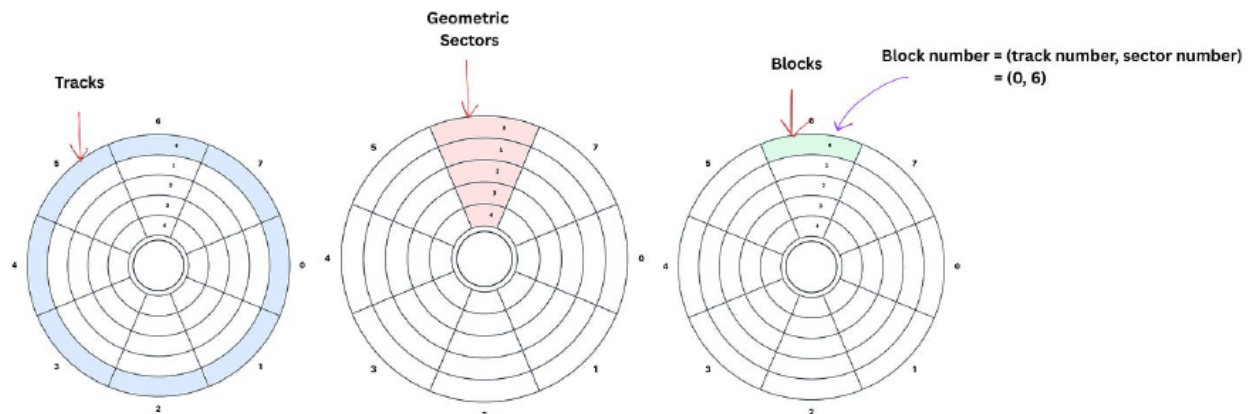
Spindle (spindle motor): Rotates the spindle, which spins all the platters at a fixed RPM. (rotations per minute)

Tracks: Individual concentric circular rings on the platter.

Sector: This is a geometric sector, meaning just a portion of a disk. (shaped like a slice of pie.)

Block (OS/filesystem block): The intersection of a track and a sector is called a block. Hence, any location on the disk, i.e., block address, can be identified by the track number and sector number. The block size is usually decided by the manufacturer. (earlier one block was around 512 bytes, now it is around 4KB+)

Arm: Moves the read/write heads inward or outward to position them over the required track and sector.



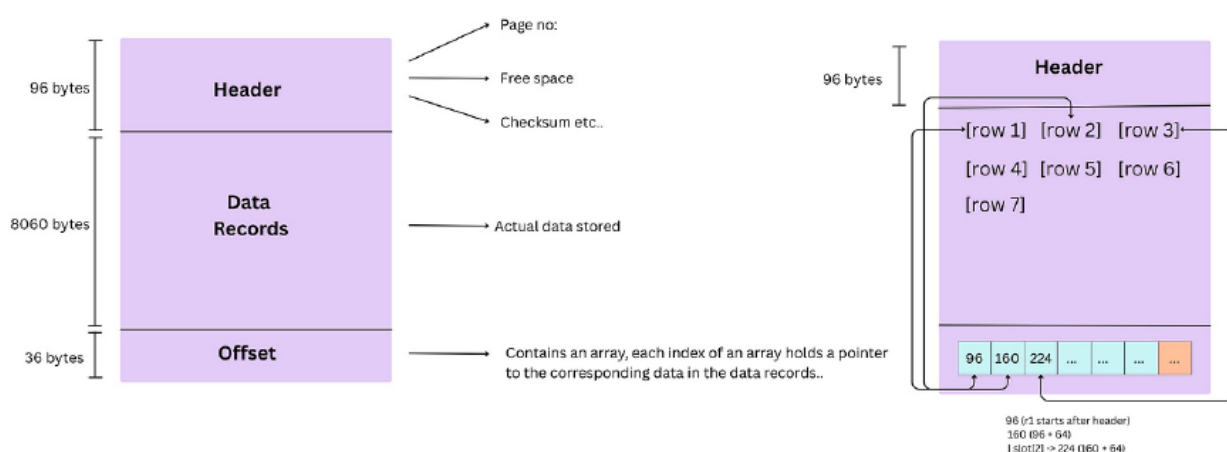
Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

How data is accessed from the disk ?

Question: So how do we access the disk to read and write? Answer : Well, we use disk APIs, which are OS-level system calls like `read()` and `write()` to read data from disk and write data back to disk.

Question : But how much data is accessed at a time? Answer: At the OS/filesystem level, it doesn't like to deal with sectors etc, so data is transferred in blocks (usually 4 KB). Even if we ask for a few bytes, the OS typically returns the bytes that contained in those 4KB blocks. Meaning it is the minimum amount of data which can be read/write by an I/O operation.

At the database level, the DB does not like to deal with these blocks, it thinks its too small a unit, so it works with pages (e.g., 8 KB / 16 KB). A page is the DB's logical unit, and it usually spans multiple blocks. So whenever a we request data (using sql query for rows), the OS ends up fetching bytes from one block or a set of blocks (depending on how much we asked for). Meaning if the disk returns bytes from 2 blocks say 8KB, then the db considers it as 1 page of 8KB. (Let's solve an example together to get a clear understanding of this..)



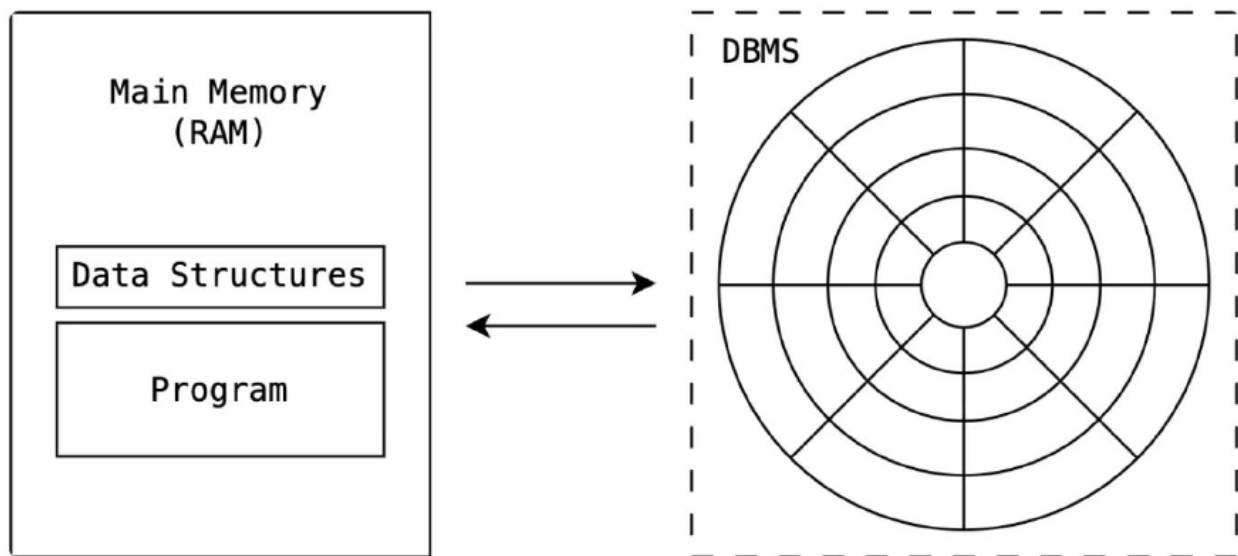
As discussed, a data page is a fixed-size chunk (e.g., 8 KB) that the database reads from disk into memory. If the underlying disk block size is 4 KB, then one 8 KB page is stored across two disk blocks, so the OS reads two blocks to bring one page into RAM.

The page starts with a header that stores metadata like page id and free space pointers.

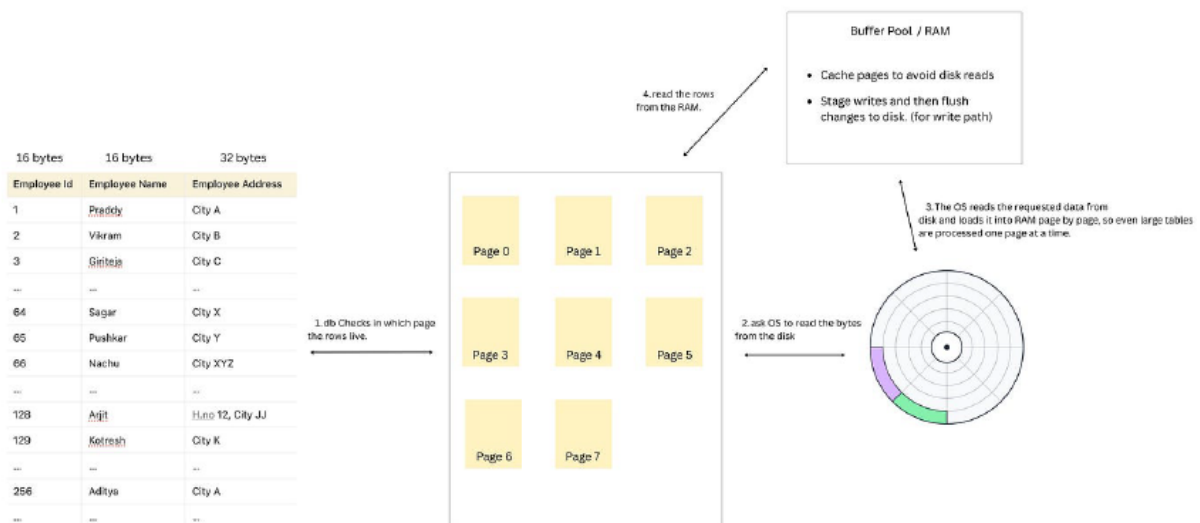
Actual rows are stored sequentially in the data area as bytes.

At the bottom of the page is a slot directory, which is an array of offsets. Each slot entry stores the byte position where a particular row begins inside the page.

Question : But after accessing the data where are we transferring it and storing it? Answer : The data that's read (the block/blocks) is first brought into our working memory (RAM). From there, the CPU can process it efficiently. And you might know that inside our working memory, we organize and manipulate that data using data structures. (arrays, hash maps, trees, etc.).



Lets try and put everything together and understand the high level flow of how data flows in and out..



Read Path

DB finds the page where the rows live and identifies offset on disk for those rows. (If an index exists, the database uses the index to directly identify the page containing the row. If no index exists, the database scans pages sequentially until it finds the row.)

DB asks the OS to read the bytes from that page file from certain offset till a certain offset.

OS checks filesystem cache; if missing, OS reads from disk and returns to the db.

DB places the page into the buffer pool (main memory) and reads the rows from that page..

Write Path

DB finds the page where the rows live and pulls it into the buffer pool..

DB updates the row in memory.

DB writes a journal/WAL entry and persists it to disk

The page stays in memory and may receive more writes before being flushed back to disk later (reducing I/Os)

Inserts/deletes follow a similar pattern (details vary)

How data is organized on a disk with an example

Lets consider our block size of the disk designed by our manufacturer to be 4KB. And the data page size of our database to be 8 KB. And we have a table of employee records containing 1000 records like this :

16 bytes			16 bytes		
			32 bytes		
Employee Id	Employee Name	Employee Address			
1	Praddy	City A	] —→ so each row totals : 64 bytes		
2	Vikram	City B			
3	Giriteja	City C			
...	...	...			
64	Sagar	City X			
65	Pushkar	City Y			
66	Nachu	City XYZ			
...	...	...			
128	Arjit	H.no 12, City JJ			
129	Kotresh	City K			
...	...	...			
256	Aditya	City A			
...	...	...			

Now, Say if we want to query and return one row (employee_id = 999) :

We considered our page size to be 8KB right, with 8060 bytes being the empty record area.

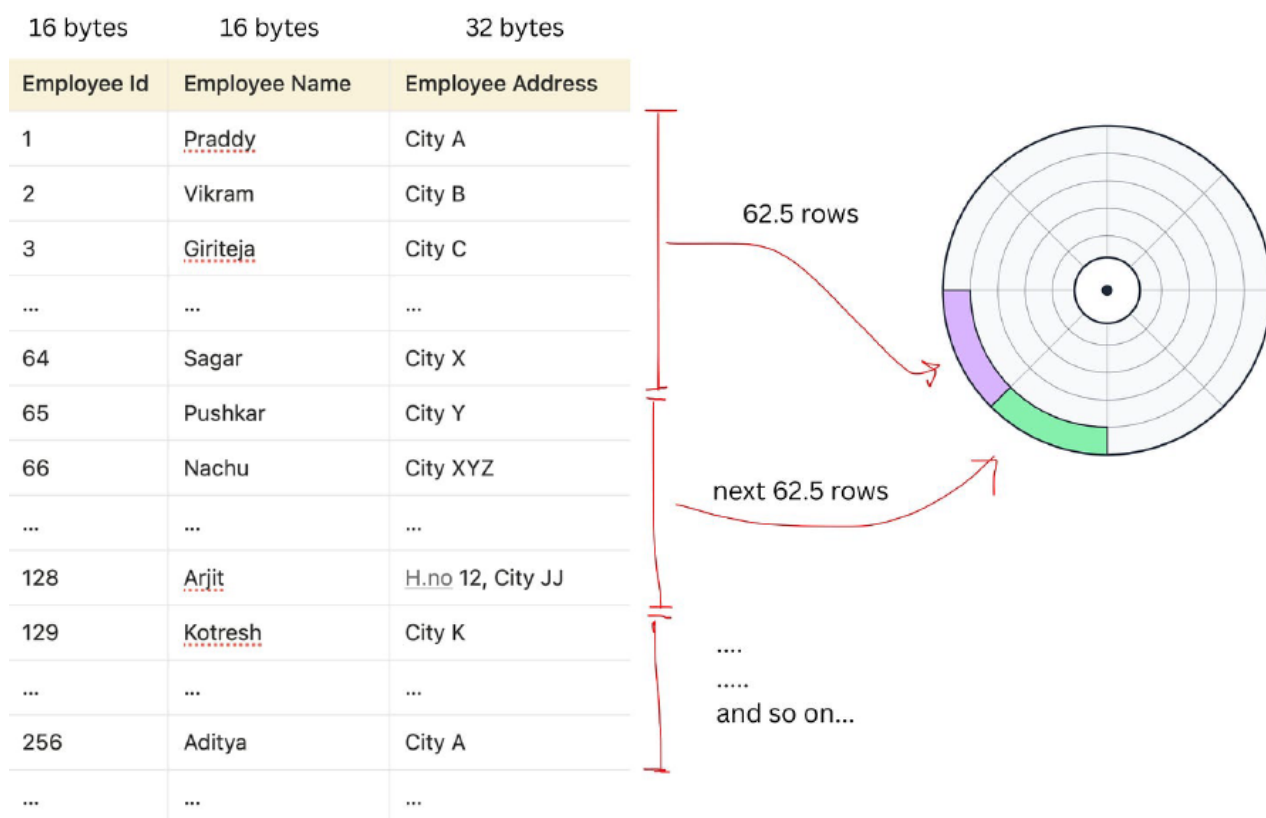
But since we are considering only 8060 bytes (as some bytes are taken away by header, and offset..), and 1 row is 64 bytes so :

So, one page will actually have 125 rows, and not 128 rows.

To cover all the rows, the total number of pages needed would be :

Rows are packed into pages; pages are fetched from disk using blocks. In order to fetch 8 pages in terms of OS level blocks :

Each 8KB page is stored on disk across 2 contiguous 4KB blocks (meaning these 125 rows are spread across 2 sequential blocks). The OS reads those 2 blocks to get back the data to the page in memory



So searching for employee_id = 999 may require scanning all 8 pages (16 blocks), even though we need only one row.

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

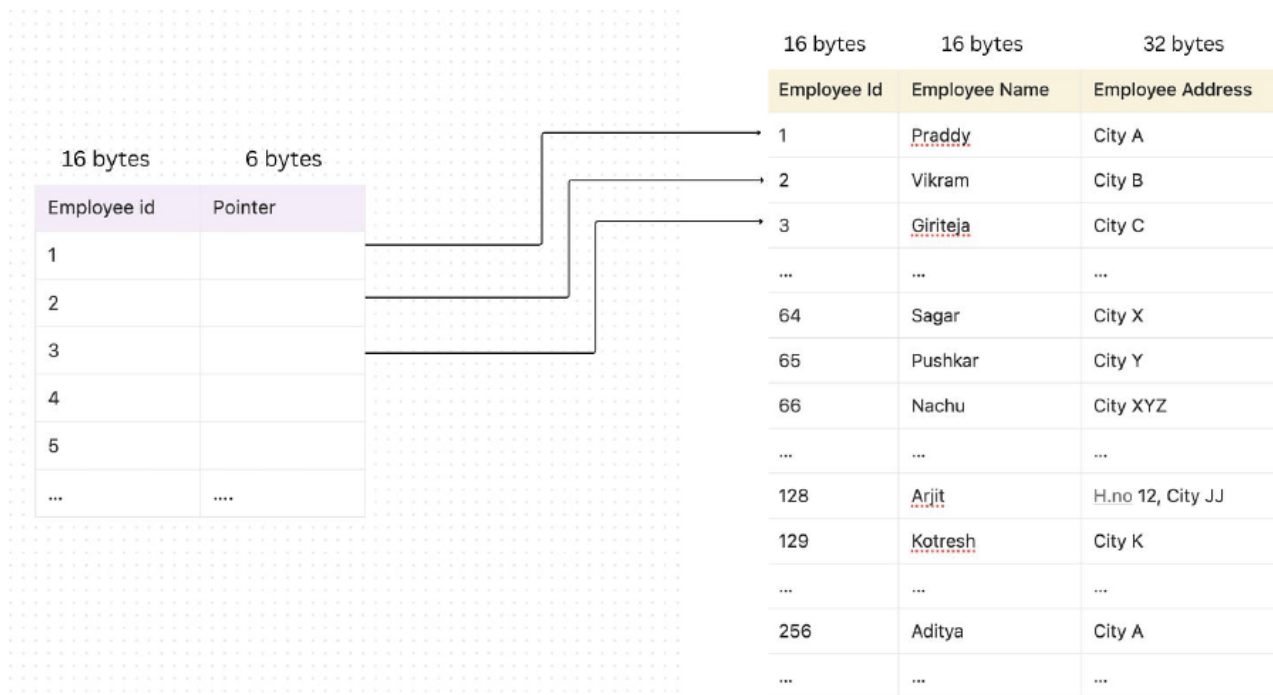
Indexing

Now imagine if we had a table of size 100000, and if had to find the last record then we had to read all the -> 800 pages, that's 1600 OS blocks worth of data to scan. As table size grows, a full scan requires reading more and more pages, which increases I/O cost.

The number of blocks we need to access is directly proportional to the read access time. So we need to reduce the number of block access.

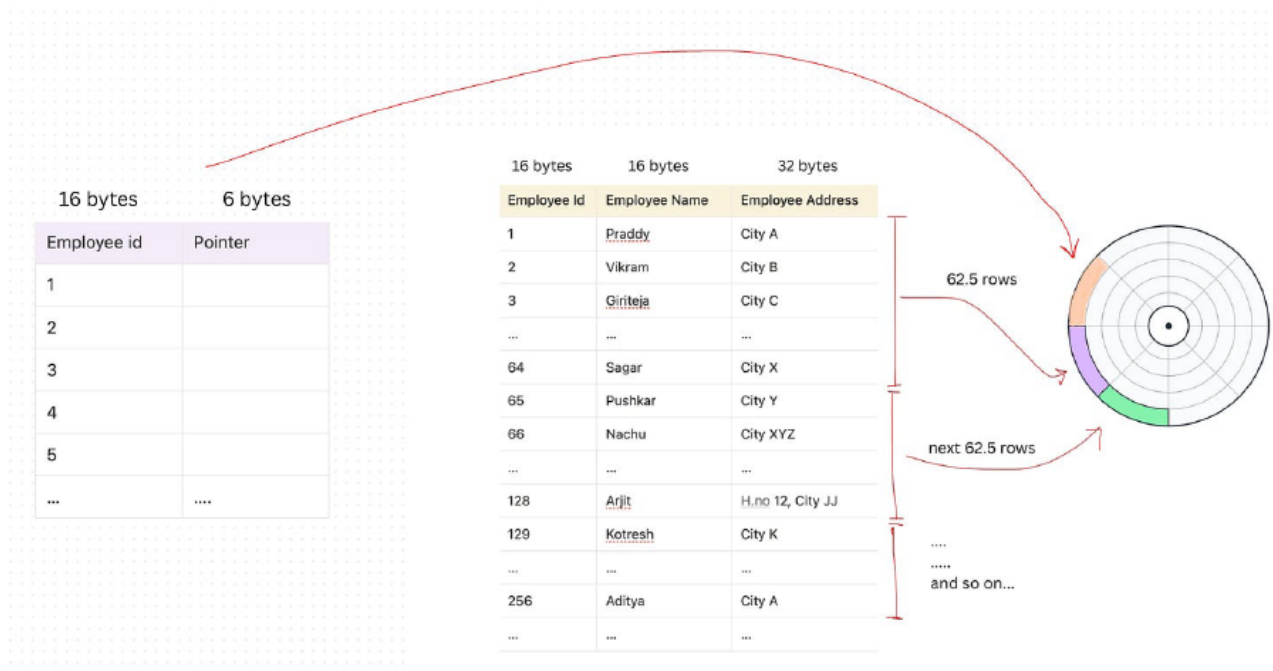
That's where indexing comes in..

Let's index the table on employee id



The next question comes as to where would we store the index table. ?

The index table also would be stored on the disk.. , so how many blocks would it take.?



So the index table would take at most 6 OS blocks.

Now the best part is: to access the index, at maximum we may read 6 blocks (3 pages). Once we find the key, we get its pointer and in a real database that pointer usually does not point directly to the “row object.” Instead, it stores something like:

Meaning:

Jump to the correct data page that contains the row

Use the page's slot directory to locate the exact row inside that page

So after the index lookup, we only need to read 1 data page (~= 2 OS blocks) to fetch the row.

So the total blocks needed becomes:

So the drastic change we saw is: without index we may read 16 blocks (8 pages), and with an index it may drop to 8 blocks (~4 DB pages).

Multi-Level Indexing

So far, our index only had 1,000 entries.

It fit into just 3 pages (6 blocks).

But what if the table had 1 million records?

Now the index would also grow proportionally.

Now imagine searching for an employee ID inside such a large index.

If we had to scan the entire index page by page, we would again be reading thousands of blocks.

So the index itself becomes expensive to search.

So to solve this, we don't scan the entire index.

Instead, we build an index on top of the index.

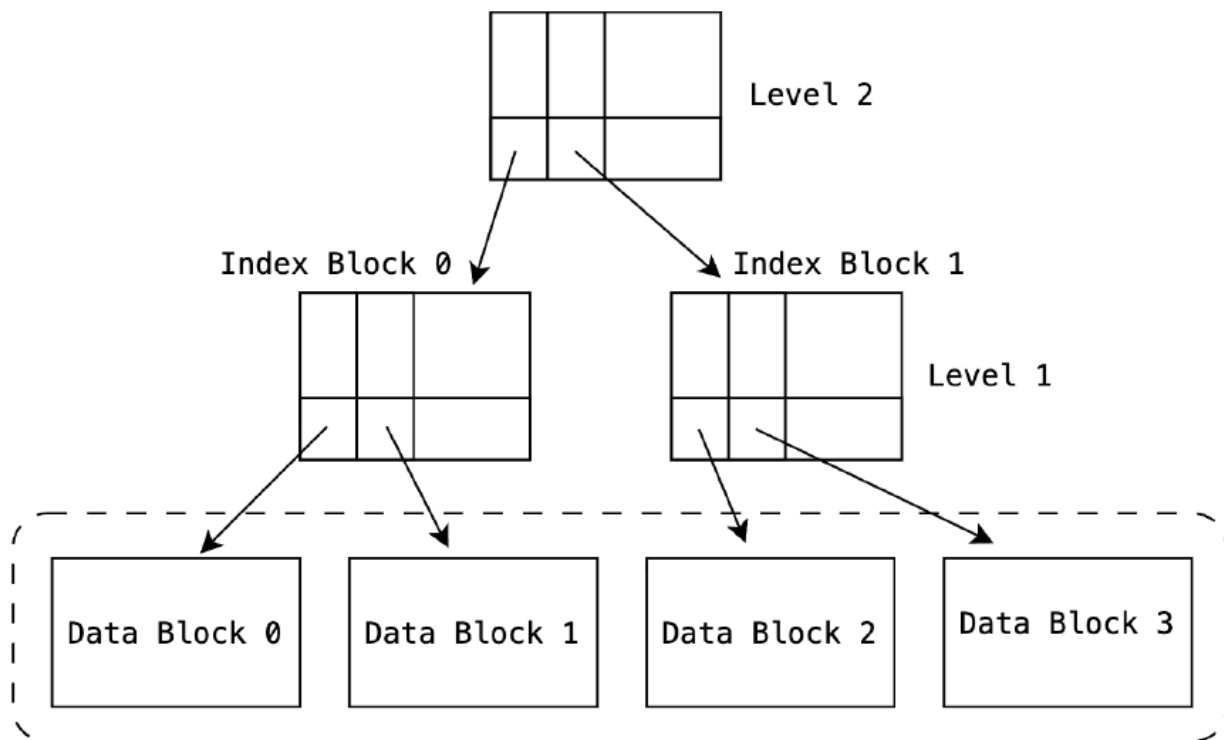
Think of it like this:

We group the 3,000 index pages into ranges.

Then we create a small “guide index” that stores:

First employee_id in each index page

Pointer to that index page



Now instead of scanning 3,000 index pages:

We first read the small guide index.

It tells us which index page to go to.

Then we read that specific index page.

Then we read the actual data page.

So instead of scanning thousands of pages, we now read only:

1 guide index page

1 lower-level index page

1 data page

That's just 3 pages total.

If we keep adding more records, we can keep adding more "guide levels".

This naturally forms a tree-like structure.

Instead of manually managing multiple index layers, databases use a self-balancing tree structure that:

Automatically grows when data grows

Automatically reorganizes itself

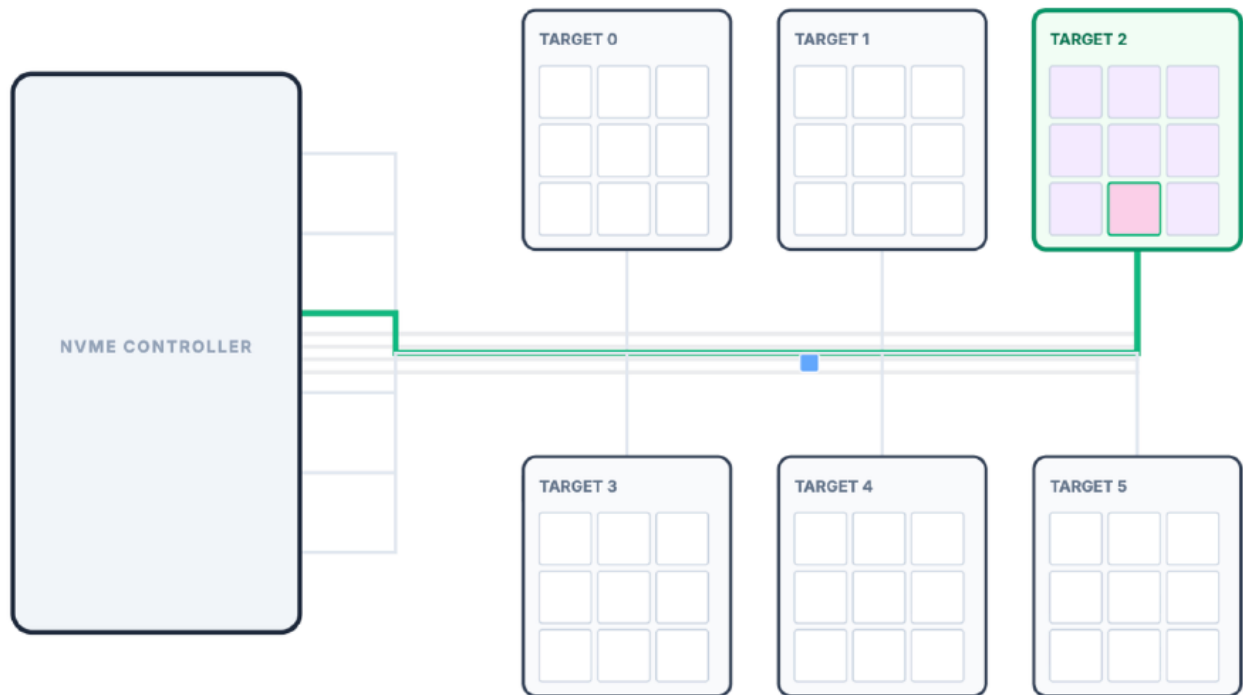
Keeps search cost logarithmic

This structure is called a B-tree (or more commonly in databases, a B+ tree).

We will talk about B- trees and B+ trees in the next articles..

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

How the data storage in SSD works ?



In SSDs, there are no spinning platters. Instead, data is stored across multiple NAND flash chips (often exposed as separate targets inside the SSD) and managed by an NVMe controller (as shown above). The important thing is: even on SSDs, the OS still talks to storage in terms of fixed-size blocks (e.g., 4KB) using logical block addresses.

So when the database requests an 8KB page, the OS issues reads for the corresponding 2 blocks. The NVMe controller then routes those block reads to the correct target / flash chip, fetches the data from the underlying flash pages, and returns the bytes back into RAM. From the database's point of view, the page-based model stays the same only the physical storage mechanics under the hood change.

If you liked this article, please do like, subscribe, and drop a comment to spark a discussion, so we can all learn from each other. And if you think it'll help someone, share it with your friends or on social media.

If you'd like 1:1 help (system design, backend, DSA, resume review etc.), you can book a session with me on Topmate

If you found this useful and want to support my work, you can also buy me a coffee ?
buymeacoffee.com/cpradyumnao..

I also write short form content on a daily basis on X and LinkedIn. Please do follow there too..

Thanks for reading Evolving Engineer! Subscribe for free to receive new posts and support my work.

References

UC Berkley, Introduction to Database Systems

Huessin Nasser, How do Databases store table data on the disk

Abdul Bari, Youtube Video

CMU Course, Database Systems

B-trees vs B+ trees

Scaling the Databases: Choosing the right strategy

2025-12-11 · Vertical Scaling, Horizontal Scaling: Replication, Sharding.

Source: <https://pradyumnachippigiri.substack.com/p/scaling-the-databases-choosing-the>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog -
<https://pradyumnachippigiri.dev/blogs/database-scaling>

When I was working as a Junior Software Engineer in my early days, I'd sit in meetings where people casually talked about "scalability" of our systems. I honestly thought it just meant "Can our database survive more traffic without crashing?" and left it at that.

Over time I realised people were really asking a deeper question:

As the business and the app grow, can the database not only survive, but also keep up and stay fast?

Scalability isn't only about scaling up when demand spikes. Good systems also let you scale down when traffic is low, so you're not burning money on overkill infrastructure at 2 a.m.

When a database can't scale, it usually fails in a few predictable ways:

CPU / memory get overloaded – queries slow down, timeouts appear, and the database starts thrashing.

Storage gets tight – disks hit capacity, writes fail, or you have to scramble for emergency cleanup.

The network becomes a bottleneck – too much data moving in and out, and everything feels sluggish.

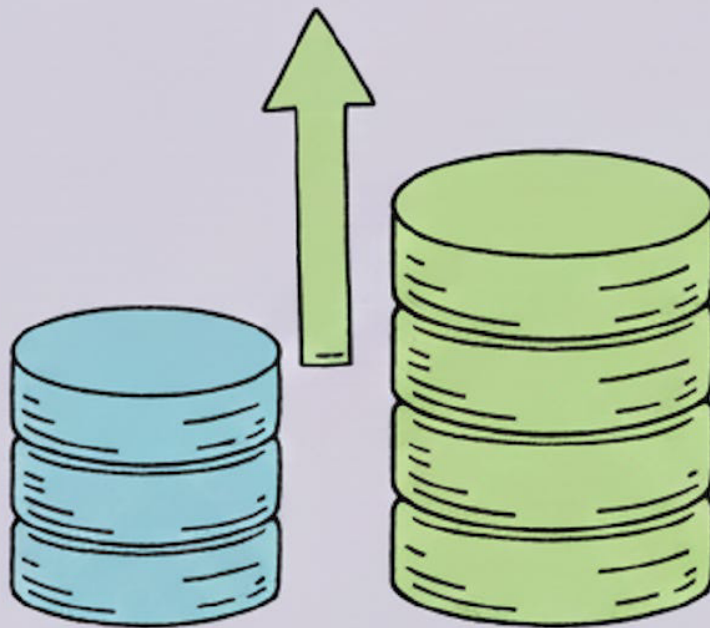
In this article, we'll walk through the core horizontal and vertical scaling strategies.

By the end, you should have a clear mental model of when to use each approach and how they fit together as your app and traffic grow.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

What is Vertical Scaling?

Vertical Scaling (Scale Up)



Vertical Scaling is the process of increasing the power/capacity (CPU, RAM, storage) of one server. In the context of database it is about making one database machine more powerful instead of adding more machines.

Instead of adding more servers, you:

Increase CPU

Add more RAM

Upgrade storage (faster disks / more capacity)

Improve network bandwidth

It's a quick and easy solution when you have a smaller database. (considering that in today's day and age our infra is on cloud and we don't have to deal with all the ugliness that comes with it, as the cloud providers provide us with one click solution to upgrade the database while they handle all the complexities)

Challenges in Vertical Scaling

Hard ceiling / physical hardware limit

There's always a maximum size for a machine. At some point, you hit the largest instance your cloud provider offers (or the biggest box you can reasonably buy). After that, you have to think about horizontal scaling.

Potential downtime during upgrades

Scaling up often requires a reboot or some kind of failover:

On many setups, the database has to restart on a bigger machine.

Managed services (like AWS RDS, Cloud SQL, etc.) automate this, but you still often see a short hiccup where connections drop or error out for a few seconds.

For users, this can look like a brief period where the app feels slow or returns errors.

Single point of failure

All your workload is concentrated on one main server. If that machine:

Has a hardware issue,

Crashes, or

Needs emergency maintenance,

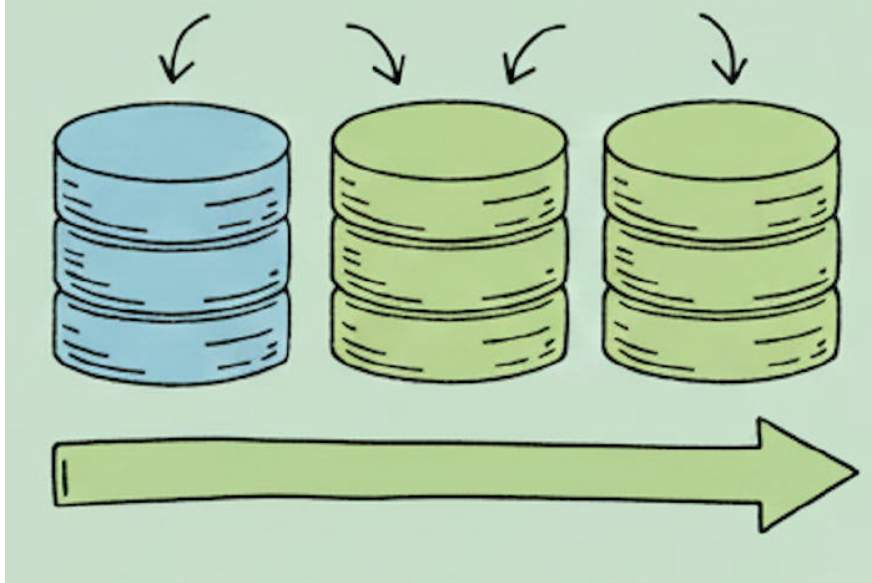
your database can go down until it's fixed or a backup takes over.

Cost grows non-linearly

Bigger instances are often much more expensive than smaller ones. At some point, it can be cheaper (and more scalable) to add more machines (horizontal scaling) instead of endlessly upgrading a single monster server.

What is Horizontal Scaling?

Horizontal Scaling (Scale Out)



Horizontal scaling is the process of adding more machines and spreading the workload across them and in the context of db it is spreading the database workload across them.

In databases, this usually shows up as:

Read replicas

Shards

Split workloads

The natural question for anybody to ask would be (which even i asked myself), why would horizontal scaling in database show up like that and why not similar to the Horizontal Scaling in Backend Services ? And that's a very good question.

For backend services, horizontal scaling is simple: you add more app servers behind a load balancer, and any server can handle any request. The servers are mostly stateless, so it doesn't really matter which one you hit.

Databases are different because they store the actual truth of your data.

You can't just spin up 5 separate database servers and let all of them accept writes on their own. Very quickly, they would disagree about the data meaning the data would not be consistent across all these servers that way, for example, one server might think your balance is Rs.500, another might think it's Rs.700. We want the database to be consistent, meaning every user and every server should see the same "truth."

So when we add more database servers, we have to decide how they share data and who is allowed to do what, and while scaling a database we should scale both the reads and the writes:

Sometimes we add read replicas -> one primary server handles writes, replicas handle reads. (scaling the reads)

Sometimes we shard the data -> each server holds a different slice of the data and only answers queries for that slice. (scaling the writes)

Horizontal scaling for databases is still “add more servers,” but always with a clear plan for who stores what and who handles which reads/writes.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Horizontal Scaling: Read Replicas

Read replicas are a standard way to scale reads from a database.

In most real-world systems, the read : write ratio is heavily skewed towards reads. So instead of letting one database handle both reads and writes, we separate them:

One database is the primary/master – it accepts all writes.

One or more read replicas called the slaves/followers are read-only and they handle only SELECT queries.

This way, we can scale reads independently by adding more replicas as traffic grows.

How do reads and writes go to different databases?

This does not happen automatically at the database level.

The application / API layer is responsible for routing:

You typically have two connection objects / pools:

One for the primary (reads + writes, or sometimes writes only)

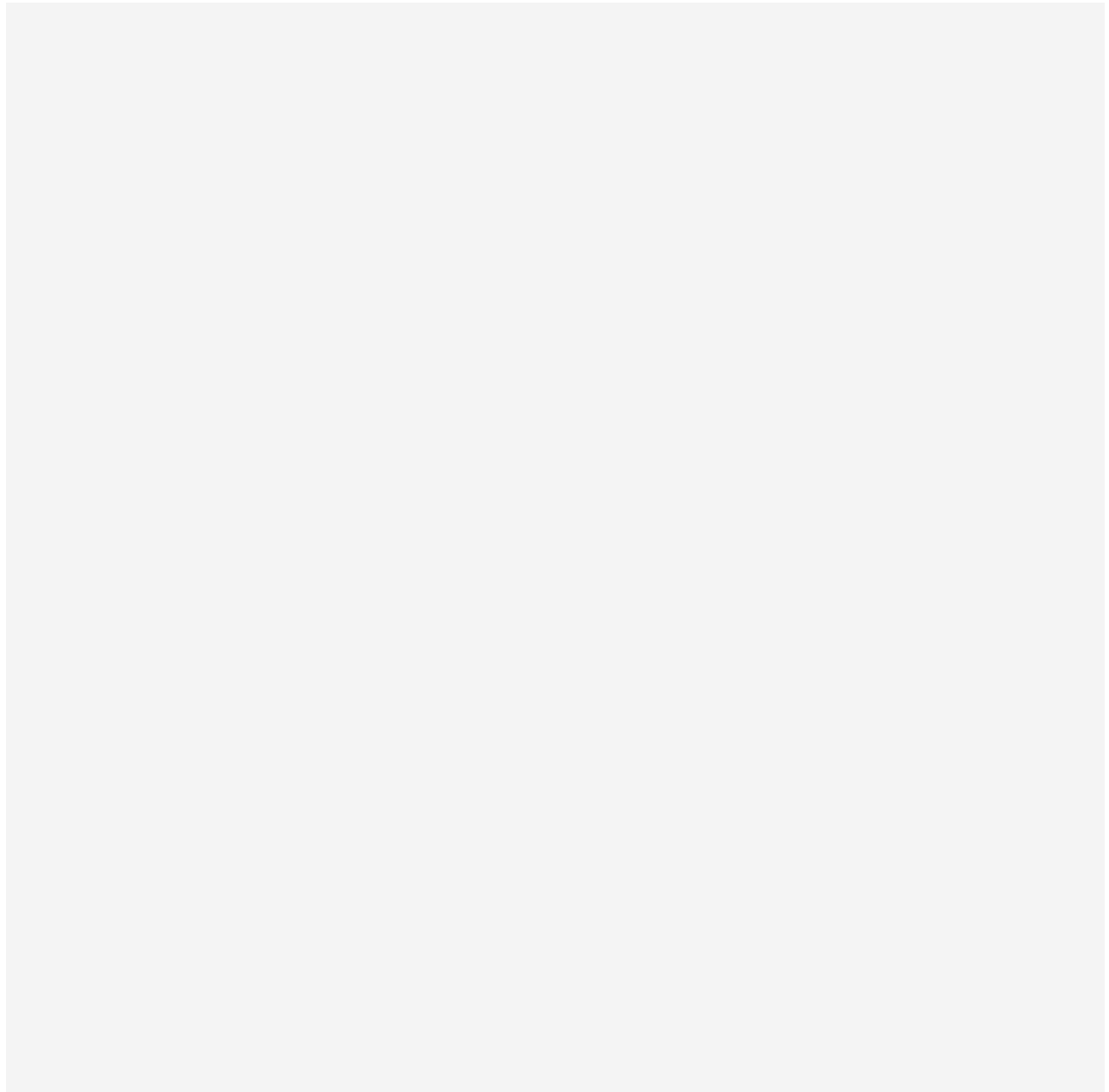
One for the read replicas (reads only)

In the business logic, you decide:

“This is a read query -> use the replica connection.”

“This is a write/update query -> use the primary connection.”

Just for example in the application layer it may look like this:



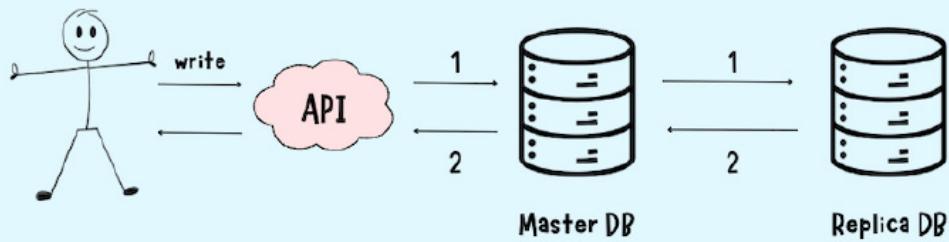
But what about consistency?

Good question: if all writes go to the primary, how do replicas see the latest data?

Changes made on the primary must be replicated to the replicas. This is done through replication, and there are two common modes:

1. Synchronous replication

Synchronous Replication



Written to master and replica before ACK



Strong consistency



Zero replication lag



Slower writes, more latency

When a write comes to the API:

The API sends the write to the primary.

The primary sends the change to the replica(s).

The write is considered successful only after the replica(s) confirm the update.

The API will not respond "OK" until both primary and replica have written the data.

Pros:

Very strong consistency – replicas are always up to date.

Replication lag ≈ 0 (no measurable delay between primary and replica).

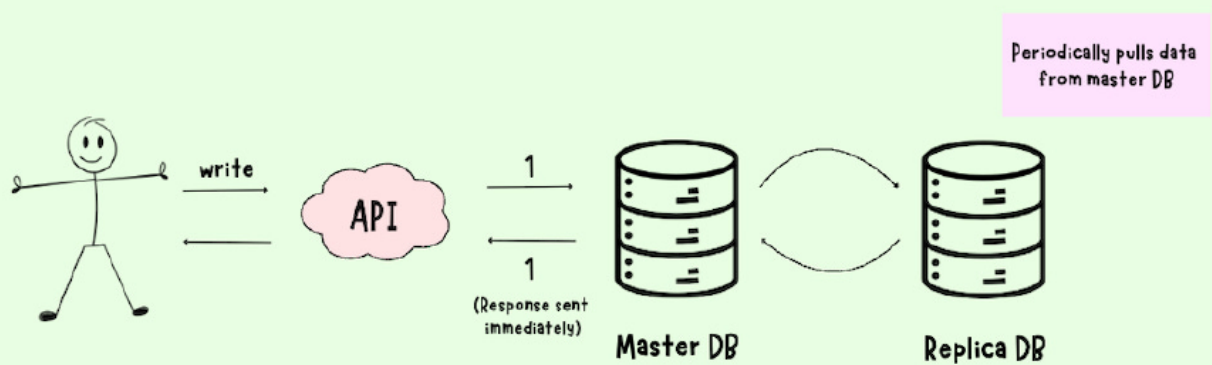
Cons:

Slower writes (you're effectively writing to multiple databases on every request).

If a replica is slow or down, it can affect write latency/availability.

2. Asynchronous replication

Asynchronous Replication



Master returns response immediately



Eventual consistency



Some replication lag



Faster writes, lower latency

This is what most production systems use by default.

The primary commits the write and immediately acknowledges success to the API.

Replicas pull changes from the primary later (via logs, binlog, WAL, etc.).

The replica applies those changes on its own copy of the data.

Pros:

Writes are fast – you only block on the primary.

System is more available even if a replica is slow.

Cons:

There is replication lag – a short window where:

The primary has the new value.

The replica still has the old value.

Reads from replicas may see slightly stale data.

This is the trade-off: strong consistency vs. performance and availability.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Horizontal scaling : Sharding

Problem: We already used read replicas to scale reads. But if one primary node still has to handle all writes, it becomes a bottleneck.

Idea: Split the data horizontally across multiple databases (shards), so writes are spread out instead of hitting a single node.

A shard is just a subset of your data stored in a separate database. Example:

Shard 1 -> Users with IDs 1–1,000,000

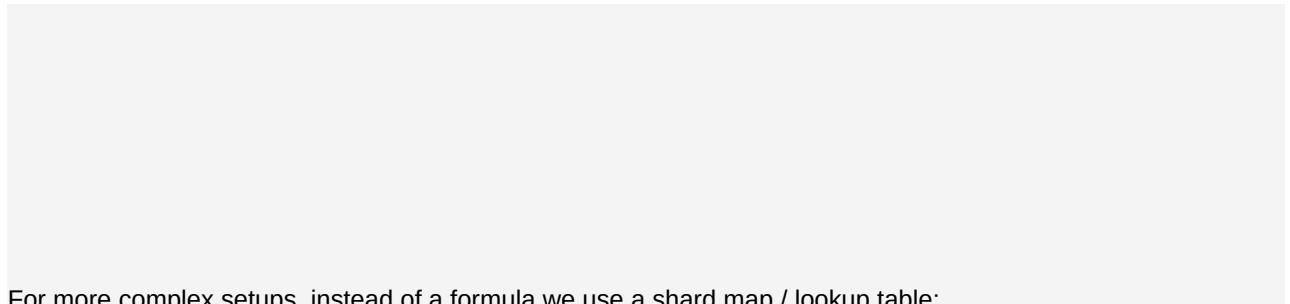
Shard 2 -> Users with IDs 1,000,001–2,000,000

You can (and usually should) have replication inside each shard:

Shard 1 primary + read replicas

Shard 2 primary + read replicas So you end up with: sharding for scaling writes, replication for High Availability + read scaling.

The decision of which shard to hit is made in the API / application layer. We define a routing strategy in our code, and every read/write request goes through that logic.



For more complex setups, instead of a formula we use a shard map / lookup table:

Key -> shard mapping is stored in a table or config (e.g., tenant -> shard).

The API first looks up the shard id, then uses the appropriate DB connection.

In both cases, the router is just application logic (or a small routing service) that maps:

business key -> shard id -> DB connection

This brings us to the end of this article. I hope you were able to take away something useful and build a clearer mental model of how databases scale.

In the next part, we'll go much deeper into sharding: we'll look at different sharding strategies, how sharding compares to partitioning, and explore even more patterns to scale reads and scale writes as your system grows.

If you liked this article then please do like and subscribe. See you in the next one.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Nail Sharding in System Design Interviews: Range-Based, Hash-Based, Directory-Based

2025-12-12 · A Practical guide about Database Sharding and Partitioning in System Design Interviews

Source: <https://pradyumnachippigiri.substack.com/p/nail-sharding-in-system-design-interviews>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/sharding>

Previously, we talked about how we deal with a situation when an application grows overnight more users, more traffic, more queries. Our first reaction was straightforward:

scale vertically (bigger machine),

scale horizontally (more servers for the app),

add read replicas to offload read traffic.

And that worked... for reads. You can read about it here -

<https://pradyumnachippigiri.substack.com/i/181087908/horizontal-scaling-sharding>

Read replicas were amazing because they helped us spread the read load across multiple machines. But there was one thing we still couldn't scale: Writes.

No matter how many replicas we added, every single write still went to one place the primary database.

All inserts? -> Primary

All updates? -> Primary

All deletes? -> Primary

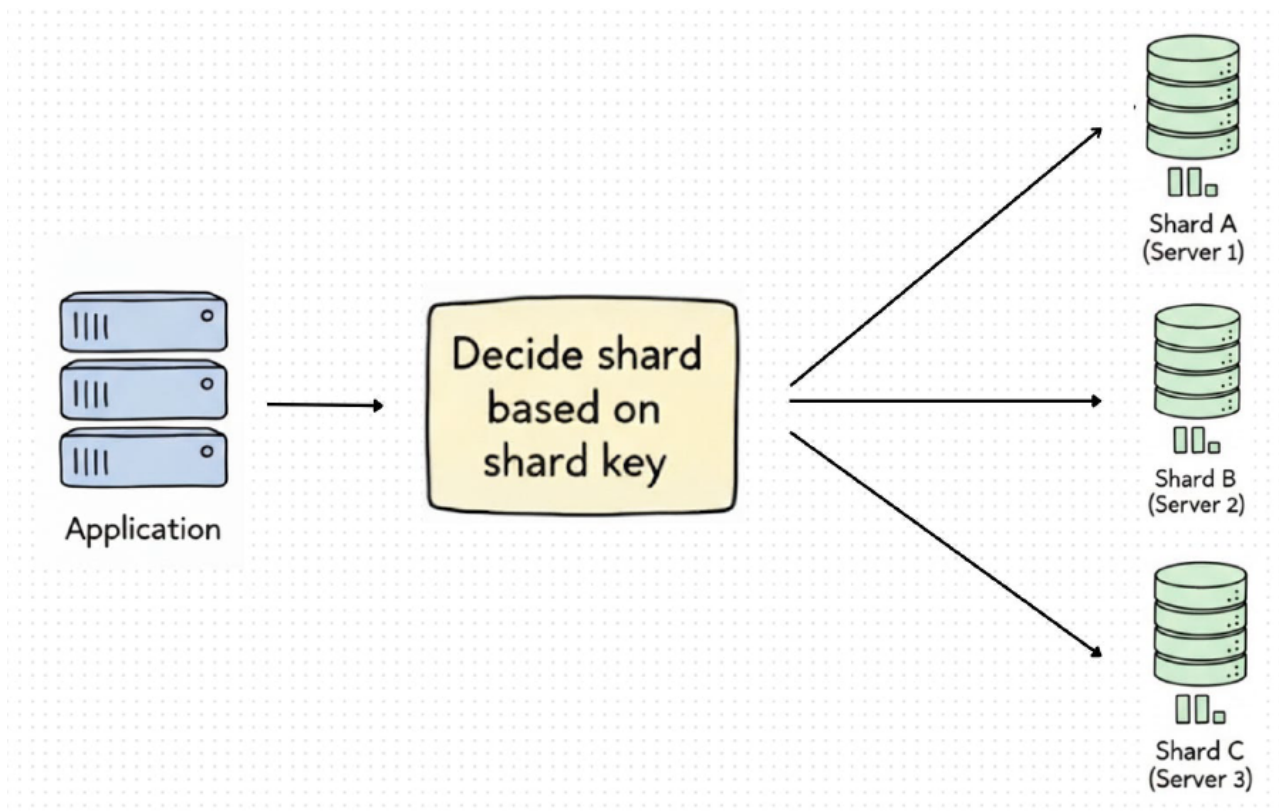
And sure, we can increase CPU, add RAM, use faster disks... But hardware scaling hits a limit. Eventually, this one machine becomes the bottleneck for write heavy systems.

And that's exactly where sharding comes in.

Sharding

Split the data horizontally across multiple databases (shards), so writes are spread out instead of hitting a single node. So each shard now has its own CPU, its own memory, its own storage, its own connection pool.

A shard is just a subset of your data stored in a separate database.



Example:

Shard 1 -> Users with IDs 1–1,000,000

Shard 2 -> Users with IDs 1,000,001–2,000,000

As we grow we can increase the number of shards.

You can (and usually should) have replication inside each shard:

Shard 1 primary + read replicas

Shard 2 primary + read replicas So you end up with: sharding for scaling writes, replication for High Availability + read scaling.

So Sharding solves the problem of scalability in writes, but you may ask questions like:

How does my request even know which shard to go to ?

What if one shard goes down

What if one shard receives most of the traffic?

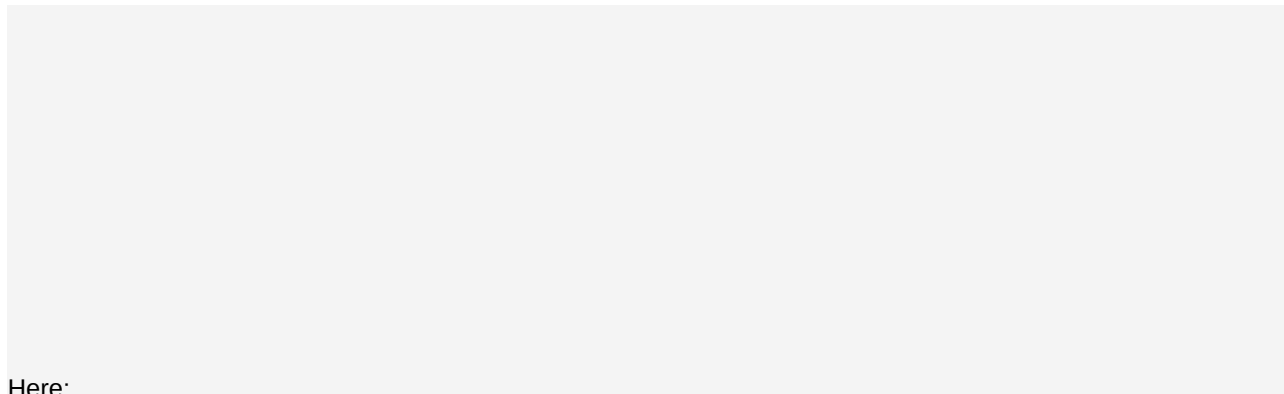
How to shard your data?

When you decide to shard, you're really making two decisions that work together:

What to shard by (the shard key): This is the field/column you use to split the data – for example, `user_id`, `tenant_id`, or `country`. It defines how the data is logically grouped.

How to distribute it (the distribution strategy): This is the rule that assigns those groups to shards. It defines how those groups are spread across machines.

In practice, this logic usually lives in your API/backend code as a routing strategy. Every read or write request goes through this routing logic before it touches the database.



Here:

Shard key = userId

Distribution strategy = $\text{userId} \% 4$ (a simple hash-based rule)

Router = this bit of code that picks the correct DB connection

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Choosing the Right Shard Key ?

When you shard a database, the shard key decides how your data and traffic are split across machines. When you shard/partition data, you expect traffic to be spread out evenly.

A shard that receives disproportionately high traffic (reads or writes) compared to others, then it becomes a bottleneck while other shards sit mostly idle. This is called as hot partition. Example:

You shard by country

60% of users are from US

The US shard is overloaded = hot

Other shards (for smaller countries) are mostly idle = cold

In that case, even if you have 10 shards, your system behaves like one overloaded shard.

So a good shard key should have:

High Cardinality

High cardinality makes it easier to avoid hot partitions, because the key space is nicely spread out.

Good: user_id, order_id, tenant_id

Bad: country, plan_type, created_at (only a few values -> some shards get all traffic)

Align with Query Patterns

Your shard key should match how your application queries data.

If most queries are WHERE user_id = ? -> shard by user_id

If you usually load "all data for a user" -> that user's data should live on one shard

If your shard key has nothing to do with your query filters, you'll end up with:

Cross-shard queries

Distributed joins

Higher latency and complexity

Immutable and Stable

The shard key should never change after it's chosen.

Good: internal IDs (user_id, tenant_id)

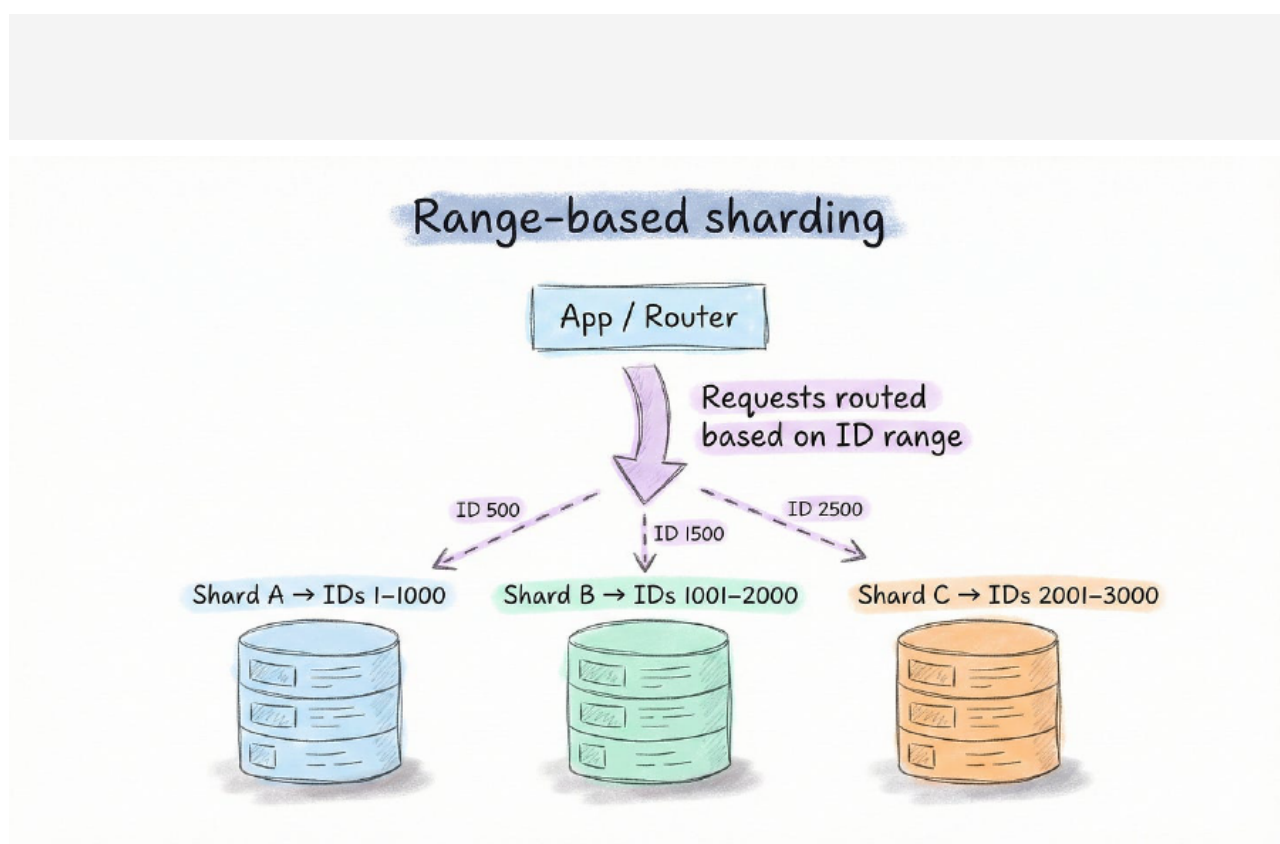
Bad: email, phone, username, status

If the shard key changes, you'd have to move rows across shards, which is slow, error-prone, and painful in production

Different Sharding Strategies

1. Range Based Sharding

Range-based sharding divides your data into contiguous ranges:

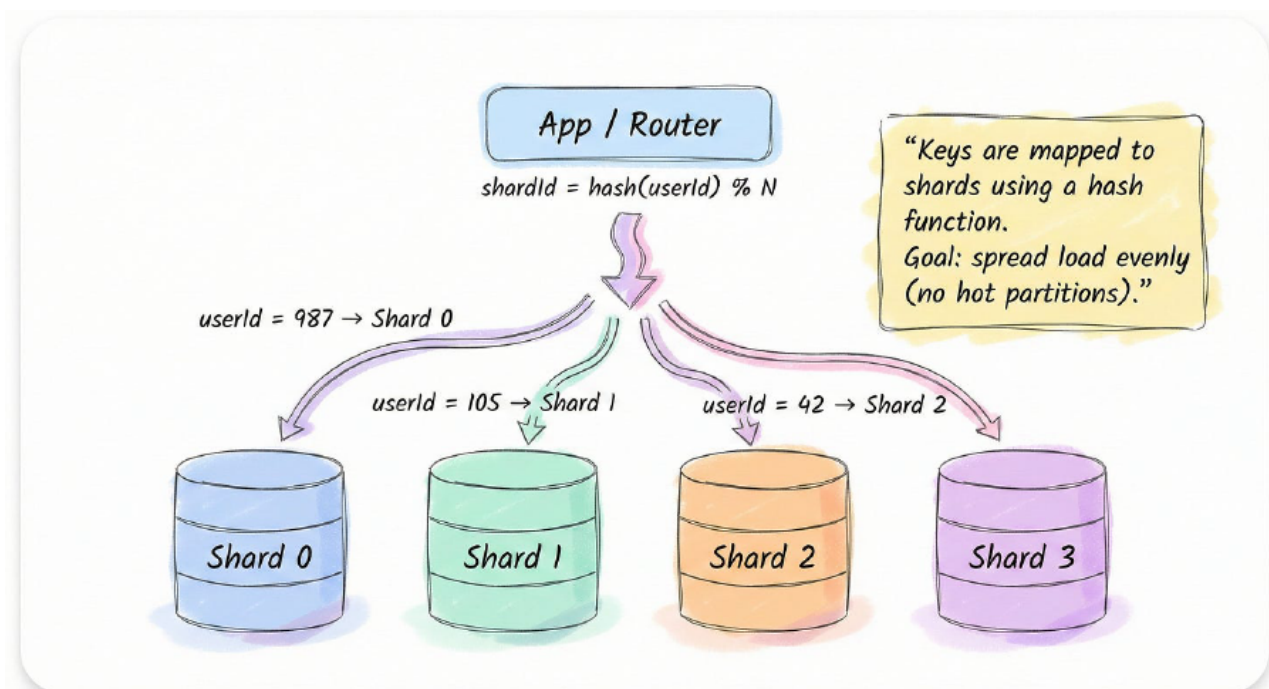


The main advantage of range-based sharding is simplicity and support for efficient range scans. If you need all orders between user IDs 500K and 600K, you only hit one shard.

Range-based sharding is simple to implement and understand but it can lead to "hot spots," where one shard is handling much more data than others. Early on, all traffic goes to the first range (because only low IDs exist), so one shard does all the work while the others are empty; later, as the system grows, all new writes and "recent activity" queries pile onto the latest range, so the last shard becomes the new hotspot.

2. Hash Based Sharding

Hash sharding uses a hash function to evenly distribute records across shards. Instead of assigning ranges, you take a shard key like user_id, hash it, and use the result to pick a shard. This is the default and most common sharding strategy.



Hashing evenly distributes data across shards, minimizing hot spots.

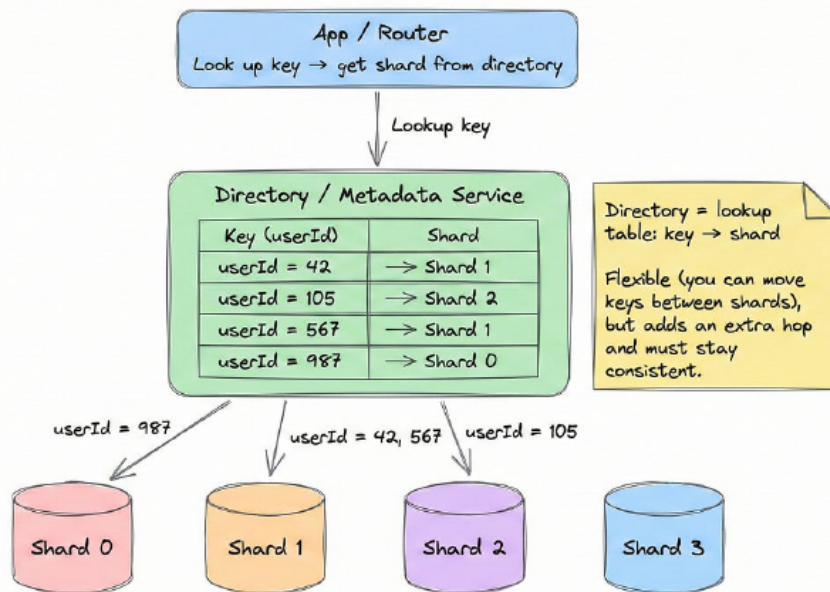
Hash-based sharding using $\text{hash}(\text{key}) \% N$ spreads data evenly at first, but the moment you add or remove a shard, the value of N changes and almost every key now maps to a different shard, causing a massive reshuffle and painful rebalancing. Consistent hashing fixes this by placing shards and keys on a hash ring so that when the number of shards changes, only a small fraction of keys move to new shards, keeping the system much more stable and easier to scale.

We will talk about Consistent hashing in a different article.

3. Directory Based Sharding

Directory sharding uses a lookup table to decide where each record lives. Instead of using a formula, you store shard assignments in a mapping table or service.

Directory-based sharding



Directory-based sharding gives you maximum flexibility: you keep a mapping of `userId` → shard, so you can move heavy users to dedicated shards or rebalance load just by updating the directory instead of reshuffling all data. The tradeoff is that every request needs a directory lookup, which adds latency and turns the directory into a critical dependency if it goes down, your whole system is effectively down.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Partitioning

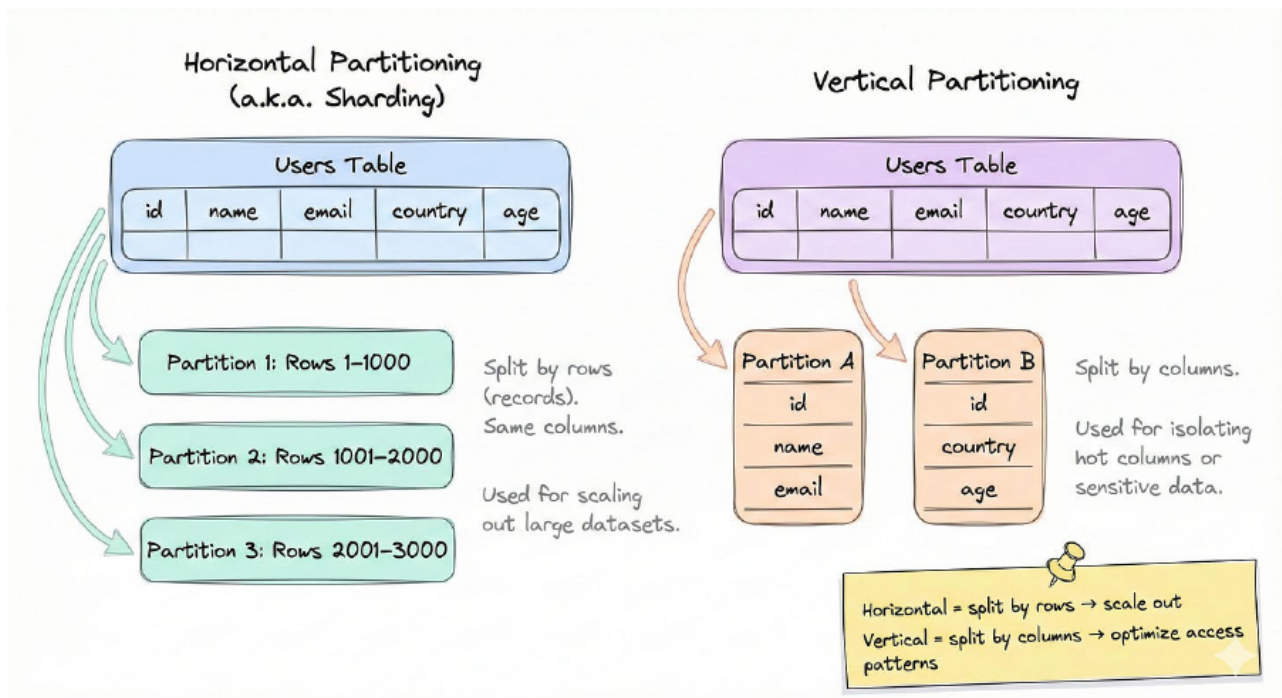
Partitioning means splitting a large table into smaller pieces inside a single database instance.

Unlike sharding, partitioning doesn't spread data across multiple machines. Instead, it helps organize data internally to speed up queries. Partitioning directly impacts performance and data manageability. Here are some of its key benefits:

Query optimization: Speeds up queries by limiting the search scope to a specific partition.

Efficient data management: Simplifies data lifecycle management by segregating data for archiving or deletion.

Better indexing and maintenance: Indexes can be applied at the partition level, reducing their size and making them easier to maintain. This keeps your database lean and responsive.



There are 2 types of partitions :

1. Horizontal Partitioning

Split rows across partitions. For example split by row range.

2. Vertical Partitioning

Split columns across partitions. For example, keep frequently accessed columns in one partition and large or rarely used columns in another. Same rows, fewer columns per partition.

We've reached the end of this super informative blog.

Last but not the least to say that :

A scalable architecture almost always combines:

Sharding for write scalability

Replication for HA + read scaling

Partitioning for internal DB performance

Hope you liked this article, if you liked it then please do like and subscribe. It really motivates me to do better and make such amazing content.

Take Care and See you in the next one.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Thanks for reading LevelYouUp! This post is public so feel free to share it.

Share

SQL vs NoSQL: A Simple Checklist to Choose the Right Database in Interviews

2025-12-18 · Don't Guess the Database choice in System Design Interviews: Use this decision checklist

Source: <https://pradyumnachippigiri.substack.com/p/sql-vs-nosql-a-simple-checklist-to>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/sql-vs-nosql>

You're in a system design interview and you reach the point where you need to decide how the system will store its data.

"What database would you use, and why?"

This is where everybody starts panicking and guessing.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

But choosing a database isn't a "SQL vs NoSQL" vibe-check. It's an engineering decision that directly impacts performance, scalability, cost, and how painful your system will be to maintain over time.

Before you design a schema, you need to decide what type of database you're designing for. That choice shapes everything that follows how you model data, how you query it, how it scales, and what trade-offs you accept.

There isn't a single "correct" database. Different database models exist because they solve different problems well. In a system design interview, what matters most is not the database you choose, but how clearly you justify it.

Understanding when to use relational, document, key-value, or graph databases and why you wouldn't use them in certain scenarios shows the interviewer that you're thinking in terms of trade-offs. Great system design isn't about perfect choices it's about making clear trade-offs and defending them with requirements.

In this article i will break down the different types of databases and their pros and cons to help you make an informed decision.

There are various factors to consider when selecting a database system. Here are steps to help you choose the appropriate database:

?????????? ???? ???? ?????????????? (What kind of data are you storing ?)

Analyze the nature of your data, including its structure, volume, and complexity.

Determine if your data is structured (relational), semistructured (like JSON or XML), or unstructured (e.g., text, images).

Consider the growth rate of your data and whether it's transactional or analytical.

????????? ???? ??? ??????: (What kind of queries you run ?)

See if your application is read heavy, write heavy, complex queries, realtime analytics, or simple CRUD operations.

????????????? ??????????????: (How will this grow?)

Determine whether you'll scale up (vertical) or scale out (horizontal), and how well the database supports sharding, read replicas, and failover since not all databases handle horizontal scaling equally.

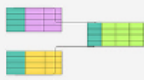
???? ??????????: (How correct does it need to be ?)

Decide whether your application requires strict ACID (Atomicity, Consistency, Isolation, Durability) compliance or if eventual consistency is acceptable.

????? ??????????: (How will you be accessing the data ?)

Consider what queries you'll run (simple lookups, joins, aggregations, text search, graph traversals) and whether the database can execute them efficiently with the right indexes and query engine capabilities.

To understand different database types, their use cases, and trade-offs, I've curated the table below to show how and when each database should be chosen. There is no one-size-fits-all solution the best database always depends on the unique requirements and constraints of your system.

Types of Databases https://pradyumnachippigiri.substack.com/						
Database Types	How data is stored	How data is stored	Core Strengths	Not good for	Best for	Examples
Relational DB		Tables (rows + columns) with a fixed schema	- ACID compliance. - complex joins and relationships.	- difficult to scale horizontally - not good for storing highly unstructured data	Transactions, finance, e-commerce, reporting	- Postgres, - MySQL, SQL Server etc
Document DB		JSON-like documents in collections (schema-flexible)	- Flexible Schema - Better at horizontal scaling than RD - A document holds all related data (denormalized) - partial updates possible - advanced indexing capabilities.	not for Complex joins and relationships.	Content/apps, profiles, catalogs, general app backends	- MongoDB - DynamoDB - Firestore
Key Value		Simple key → value pairs (often in-memory)	- simple database but ultra-fast look-up - can be heavily sharded and partitioned. - limited functionality	- not meant for complex queries - not for historical data - not for ACID transactions.	Caching, sessions, rate limiting, pub/sub	- Redis - Memcached, - dynamo db
Wide-Column		Partitioned rows in column families (denormalized). basically its a 2-D key-value store	- highly partitionable - denormalized data - best for queries by primary key - high write throughput	- not for random filtering on non-columns keys - not ACID transaction processing	Time-series, event logs, IoT, write-heavy workloads	- Cassandra - HBase
Graph DB		Nodes and edges. nodes → entities, edges → relationships	- fast multi-hop relationship traversal - no join computation. - ideal for connected data.	- difficult to scale - not for write heavy workloads	Social graphs, recommendations, fraud detection, knowledge graphs	- neo4j - neptune
Time Series DB		Time-ordered records (timestamp + metrics), usually partitioned by time	- Optimized for time-based writes and reads - Fast aggregations over time windows - High write throughput	- Complex joins - General-purpose CRUD - Highly relational data	- Metrics, monitoring - Event data - Sensor/IoT data - Analytics over time	- InfluxDB - TimescaleDB - Prometheus
Search / Index DB		Documents indexed using inverted indexes (not rows)	- Full-text search - Relevance ranking. - Fast filtering over indexed fields	- Transactions (ACID) - Being the source of truth - Heavy writes without indexing cost	- Text search - Logs & analytics search - Autocomplete & filtering	- Elasticsearch - OpenSearch - Algolia

When asked to choose a database in a system design interview, the interviewer isn't looking for a buzzword or a trendy technology. They're looking for clear reasoning.

Start with the requirements. Explain how the data is shaped, how it's accessed, how it grows, and what trade-offs matter most. Then choose a database that fits those constraints and explicitly call out what can be given up.

That's how you turn "SQL vs NoSQL?" from a panic moment into a confident discussion.

Hope you liked this deep dive and the curated table.

If you liked this article please do like and subscribe. Until then see you in the next one. ??

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Relational Databases & ACID Transactions

2025-12-02 · System Design from First Principles : Part 1

Source: <https://pradyumnachippigiri.substack.com/p/relational-databases-and-acid-transactions>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/acid-transaction>

I'm sure everyone already knows what ACID means and roughly how it works. We've all memorized Atomicity, Consistency, Isolation, Durability at some point, probably for an exam or an interview.

In this post, we're going to step back and look at the bigger picture: what relational databases are, why they exist, where they came from, and how ACID actually fits into that story. We'll go into the history, the "why" behind the design, and then break down how ACID really works in practice.

By the end, you won't just remember the acronym you'll genuinely understand what it's protecting you from and how to reason about it when you design systems.

So let's start.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Relational Databases

The database is one such component which is most brittle, which has the most latency, most response time, and anything that breaks is usually because of the DB.

Relational databases store data in tables made of rows and columns. Each table represents a type of entity (users, orders, products), each row is a single record, and each column is a field. The "relational" part comes from the fact that tables can reference each other via keys, which is what SQL is built to query efficiently.

id	name	email
1	Pradyumna	prady@example.com
2	Aditi	aditi@example.com

But do you know why relational databases need these properties in the first place? What was broken before RDMS that RDMS and ACID was invented?

History of Databases

Early on, companies stored data in files (CSVs etc). For accounting, this sucked because:

Hard to keep numbers correct when many multiple people updated the files.

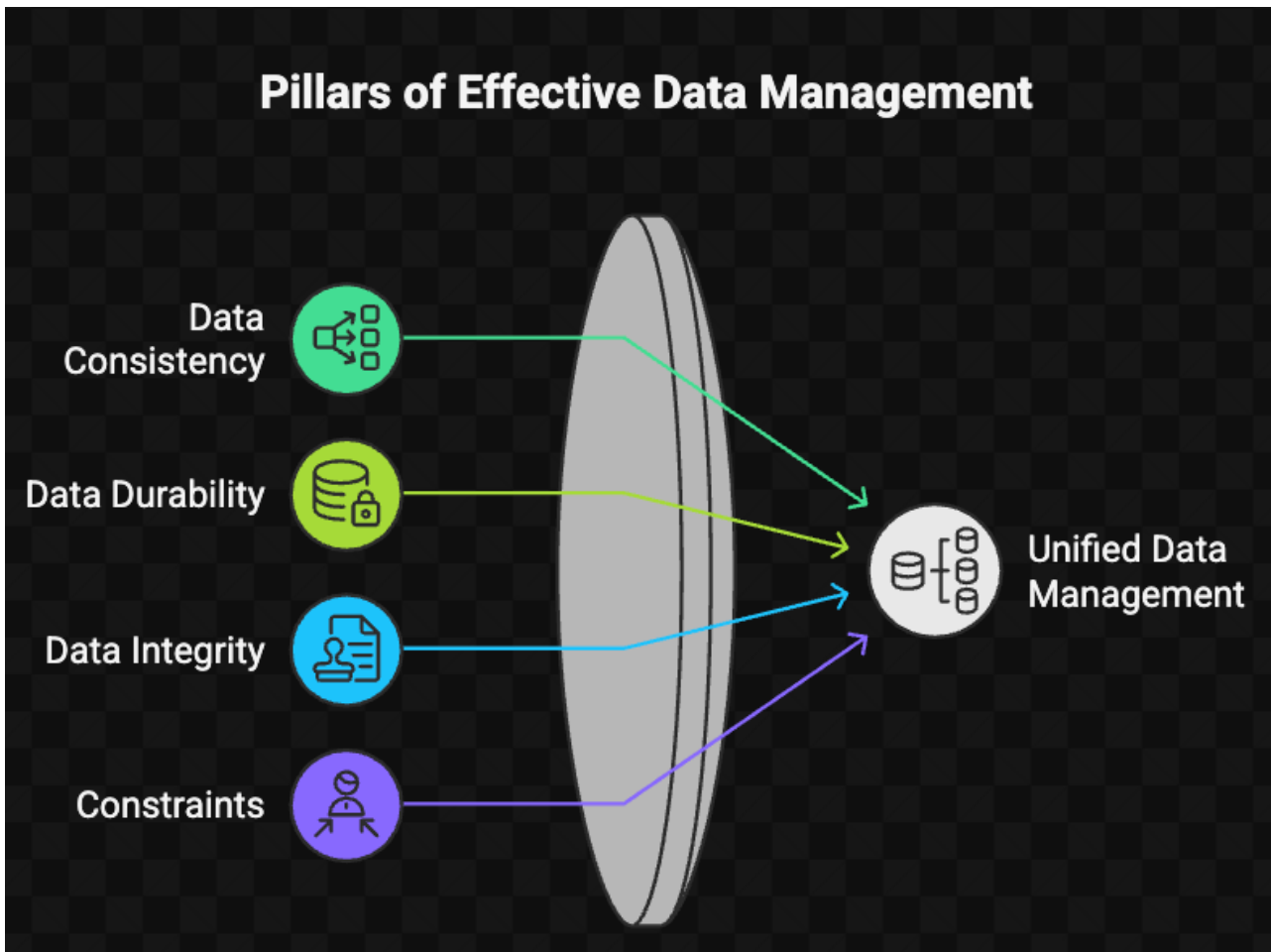
No guarantee that updates are fully applied (what if power goes off mid-write?).

No clear rules (constraints) to prevent invalid data.

Data was scattered across many files/programs.

So relational databases (RDBMS) were built to fix exactly these problems.

The key properties that the RDBMS came up with were :



And to help RDMS achieve all these properties, "Transactions" was invented.



Made with  Napkin

What is a Database Transaction?

A transaction in the context of databases is a collection of queries or a sequence of one or more operations (such as inserting, updating, or deleting records) that the database treats as one single unit of work. It either fully succeeds or fully fails, with no in-between states.

If you consider a Bank Account example, where you want to transfer a 100\$ from Account A to Account B. These are the steps you would follow right.

Transaction

ACCOUNT_ID	BALANCE
1	\$900
2	\$600

Send \$100 From Account 1 to Account 2

BEGIN TX1

SELECT BALANCE FROM ACCOUNT WHERE ID = 1

BALANCE > 100

UPDATE ACCOUNT SET BALANCE = BALANCE - 100 WHERE ID = 1

UPDATE ACCOUNT SET BALANCE = BALANCE + 100 WHERE ID = 2

COMMIT TX1

These steps form one transaction. If either step fails, both are canceled.

Without transactions, databases could end up in inconsistent states like money deducted from Account A, but not received in Account B (Partial Updates) or 2 people booking the last movie ticket at the same time (Conflict.)

Transactions solve these problems by enforcing rules like ACID properties (Atomicity, Consistency, Isolation, Durability).

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

1. Atomicity

The word atomic comes from the Greek “atomos”, meaning “indivisible, cannot be cut”. Here in the context of Transactions, Atomicity means:

All statements within a transaction takes effect or none meaning All statements within a transaction either take effect together or not at all. If even one query in the transaction fails, the entire transaction is rolled back.

Say if you had'nt implemented a Transaction and if you tried the same example of Sending 100\$ to Account B. And mid way as you update and send 100\$ to account B, the PC faces some problem and shuts down.

Atomicity

ACCOUNT_ID	BALANCE
1	\$900
2	\$500

Send \$100 From Account 1 to Account 2

BEGIN TX1

SELECT BALANCE **FROM** ACCOUNT **WHERE** ID = 1

BALANCE > 100

UPDATE ACCOUNT **SET** BALANCE = BALANCE - 100 **WHERE** ID = 1

Now you would see that the Account A has 100\$ deducted, but the Account B hasnt recieved yet, now this is a inconsistent state, hence using such queries under transactions is important. Hope you got a fair understanding of what Atomicity means.

Curious how Databases implements Atomicity ?

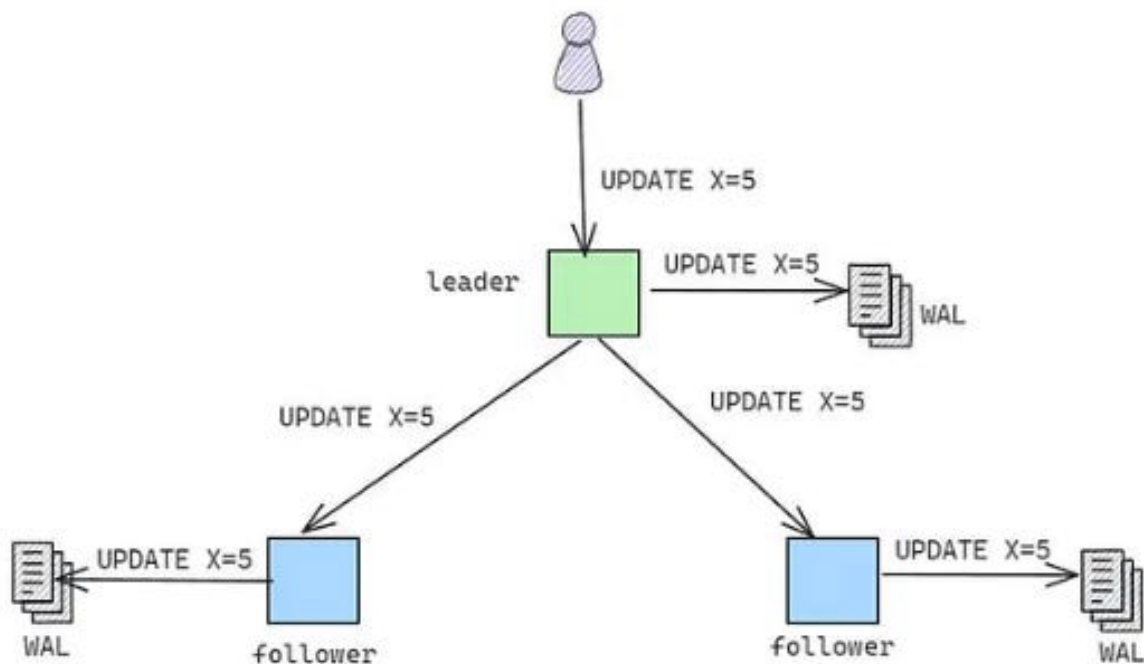
How Databases Implement Atomicity

The key mechanisms to guarantee atomicity.

Write Ahead Log (WAL) :

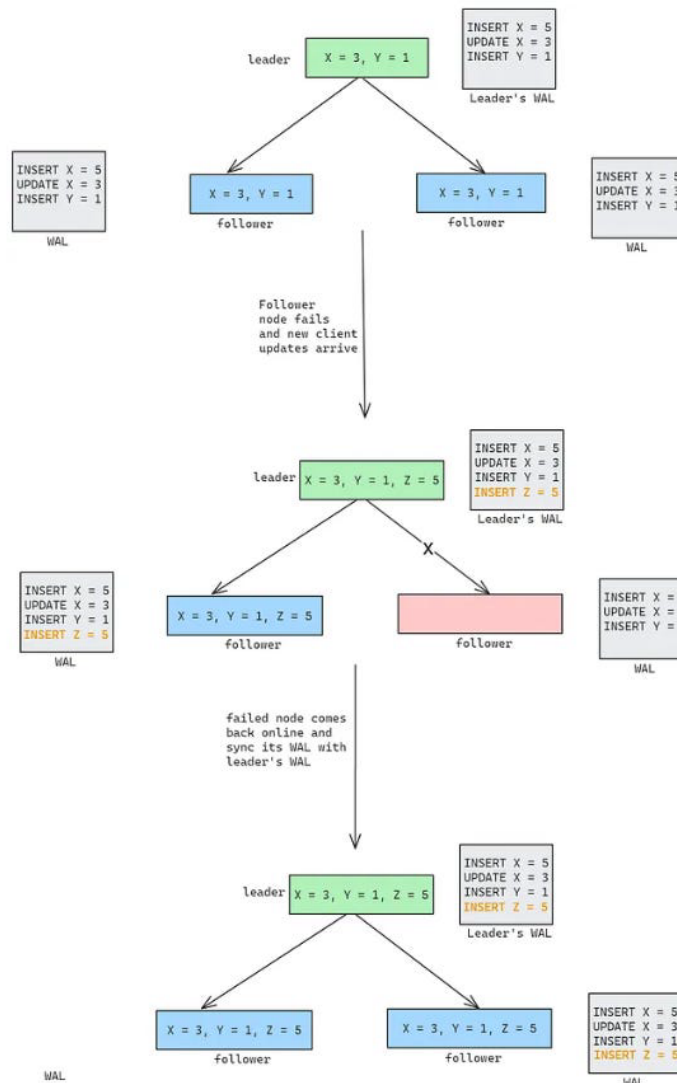
This mechanism records all database modifications before they are applied to the actual data pages.

If a transaction fails (crashes), the log provides the necessary information to undo any partial changes made, ensuring that the entire transaction is treated as if it never happened.



Write-ahead logs are written on a persistent storage (a disk) which allows them to survive a node failure. In a distributed database, once a follower node restarts after a failure, it reads its WAL file from the start and executes updates in the log file sequentially. This allows the node to return to its state before the failure. Think of it as how a bank statement lists the credit or debit transactions. If those transactions are run one by one on the initial balance, the final balance of the account can be calculated.

Once the follower node has reconstructed the data state, it synchronizes its WAL with the leader node's WAL by requesting all the client updates that happened while the node was in the failed state. After the leader node shares all the client updates, the follower node updates its WAL and executes all updates on the data to bring the data state consistent with the leader node and other follower nodes.



There can be a scenario when there are millions of entries in the WAL which can impact the restart time of a node. This is addressed by taking snapshots of the database at regular intervals. On restart after a failure, only the WAL entries that were appended after the last snapshot are executed. This greatly reduces the number of WAL entries that need to be executed to reconstruct the prior state of the database and hence improves the performance.

2. Consistency

Consistency in the context of ACID transactions ensures that any transaction will bring the database from one valid state to another valid state never leaving it in a broken or “invalid” state.

By Consistency, we refer to both types :

Consistency in reads

Consistency in Data

Lack of Atomicity and lack of isolation leads to inconsistent data.

Consistency means that all data integrity rules are true before and after a transaction. Those rules are things like:

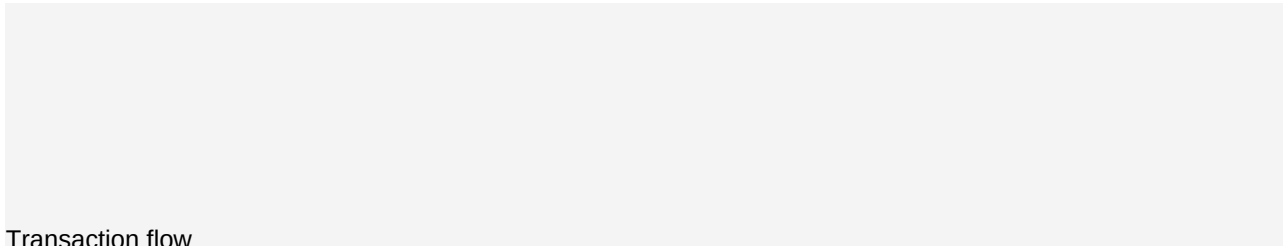
PRIMARY KEY – no duplicate IDs

FOREIGN KEY – child rows must point to existing parent rows

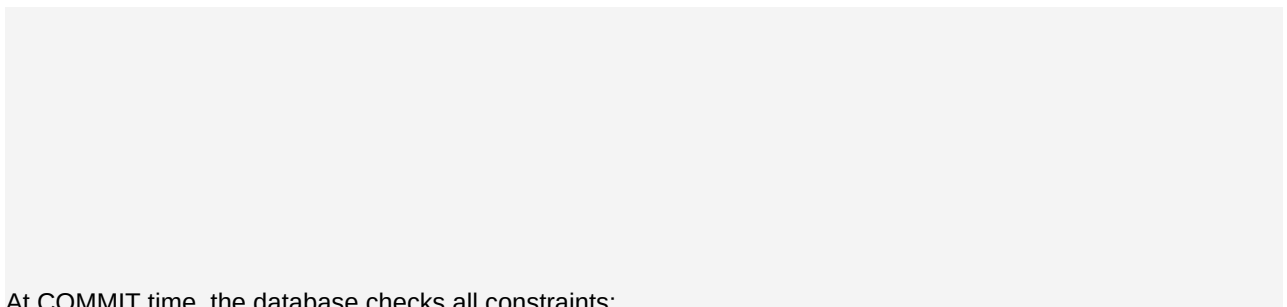
CHECK – values must satisfy conditions (e.g., age must not be negative , $\text{age} \geq 0$)

If a transaction tries to violate any of these constraints, the database simply refuses to commit it and rolls back to the previous valid state.

Say you have a users table:



Transaction flow



At COMMIT time, the database checks all constraints:

age_non_negative is violated ($-5 < 0$).

The transaction cannot leave the DB in this invalid state.

So the whole transaction is rejected/rolled back – you won't see a user with age = -5 in the table.

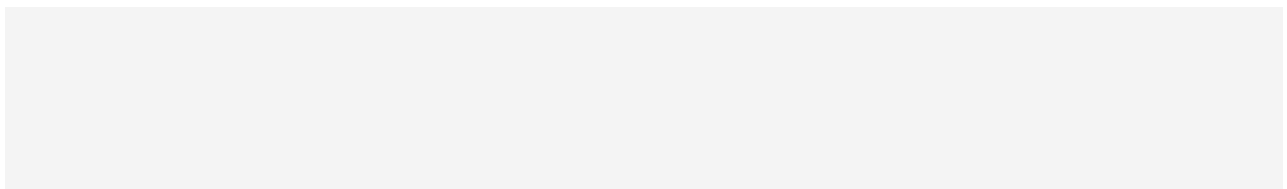
How to implement Consistency?

Atomicity is implemented by the databases itself, so there is not user involvement there, but consistency is user defined.

Database Schema Constraints : NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK constraints, and other schema definitions ensure no invalid entries are allowed. Like we saw in the previous example.

Triggers and Stored procedures (You need not understand the code below, it is just for a higher level understanding)

Trigger example – prevent negative stock on update



Any UPDATE products that would make stock_quantity < 0 is automatically rejected.

Stored procedure example

Call:

The procedure centralizes validation + updates -> fewer chances for apps to misuse the DB.

Application level Safeguards : The application should still protect the database from obviously bad input, by providing with robust validation in the backend code before any operation on the DB.

In the next article we will go in depth with hands on to understand Isolation and Durability. This article may become long incase i try to cover all of them . Sorry about that. I hope you liked the article, if you did, please like and subscribe.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Take Care.

See you in the next one ??

Relational Database & ACID Transactions : Part 2

2025-12-05 · System Design from First Principles : Part 2

Source: <https://pradyumnachippigiri.substack.com/p/relational-database-and-acid-transactions>

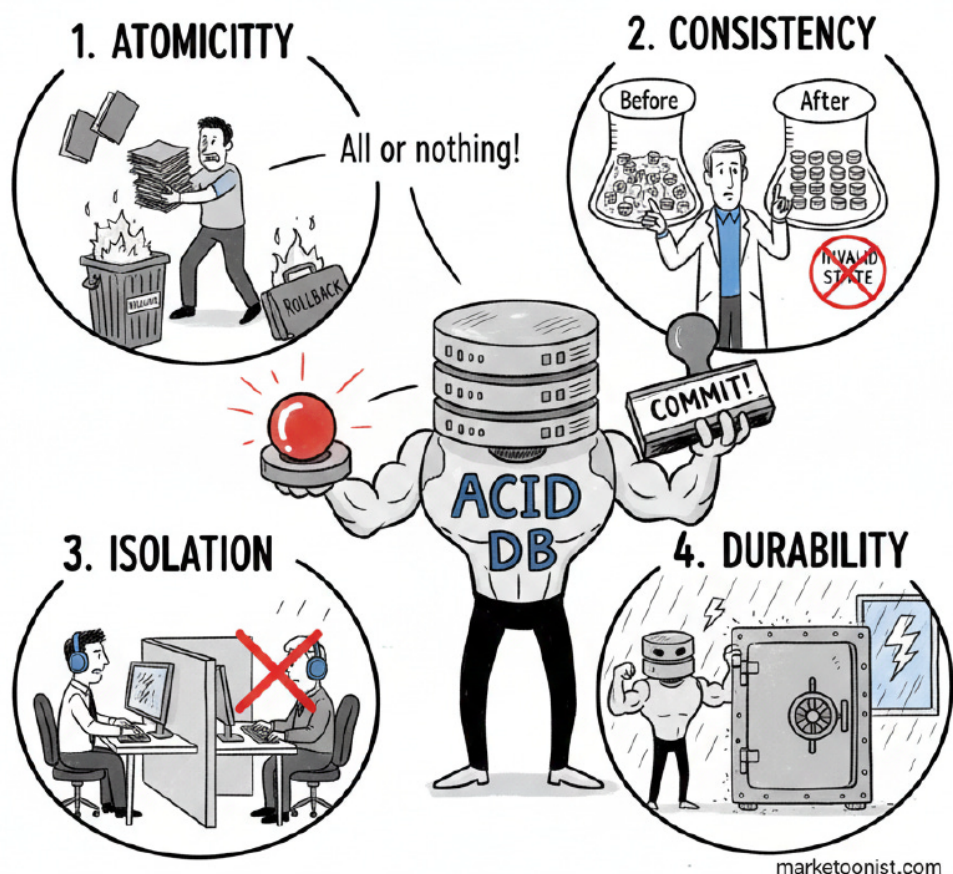
Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/acid-transaction-part2>

In Part 1 of the System Design from first principles series we discussed about Relational Databases, their history and the “A” and “C” of ACID.

In this one, we will continue the same and discuss more about the “I” and “D” in ACID with some practical examples as well.

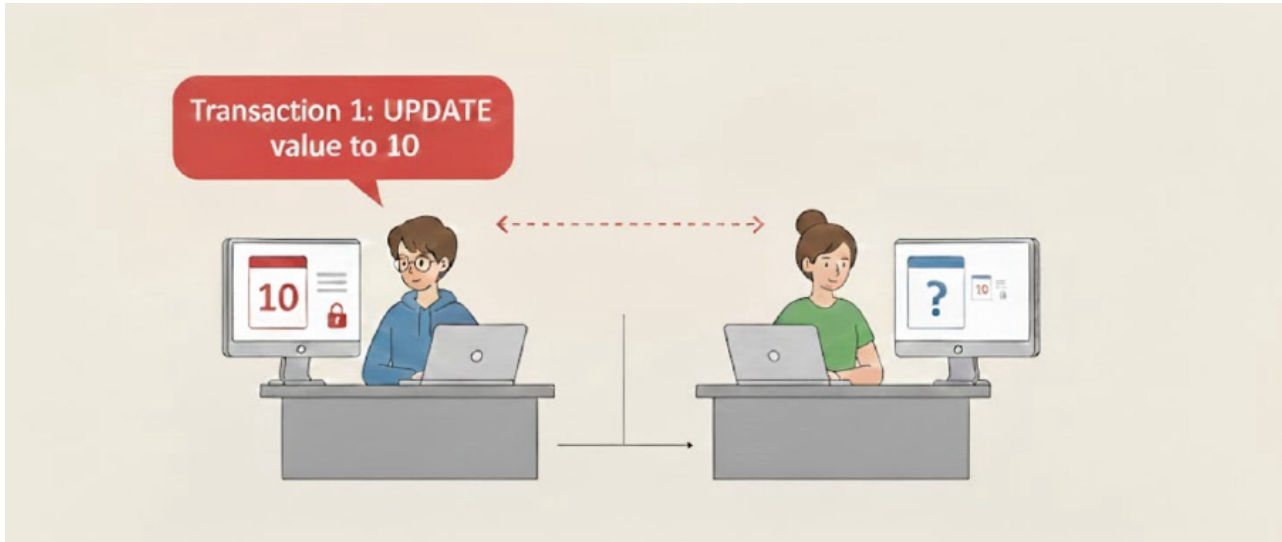
Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.



ACID TRANSACTIONS: DATA YOU CAN TRUST!

3. Isolation

When multiple transactions are executing in parallel, the isolation levels determine how much changes of one transaction are visible to others. Isolation ensures that concurrently running transactions do not interfere with each other's intermediate states.



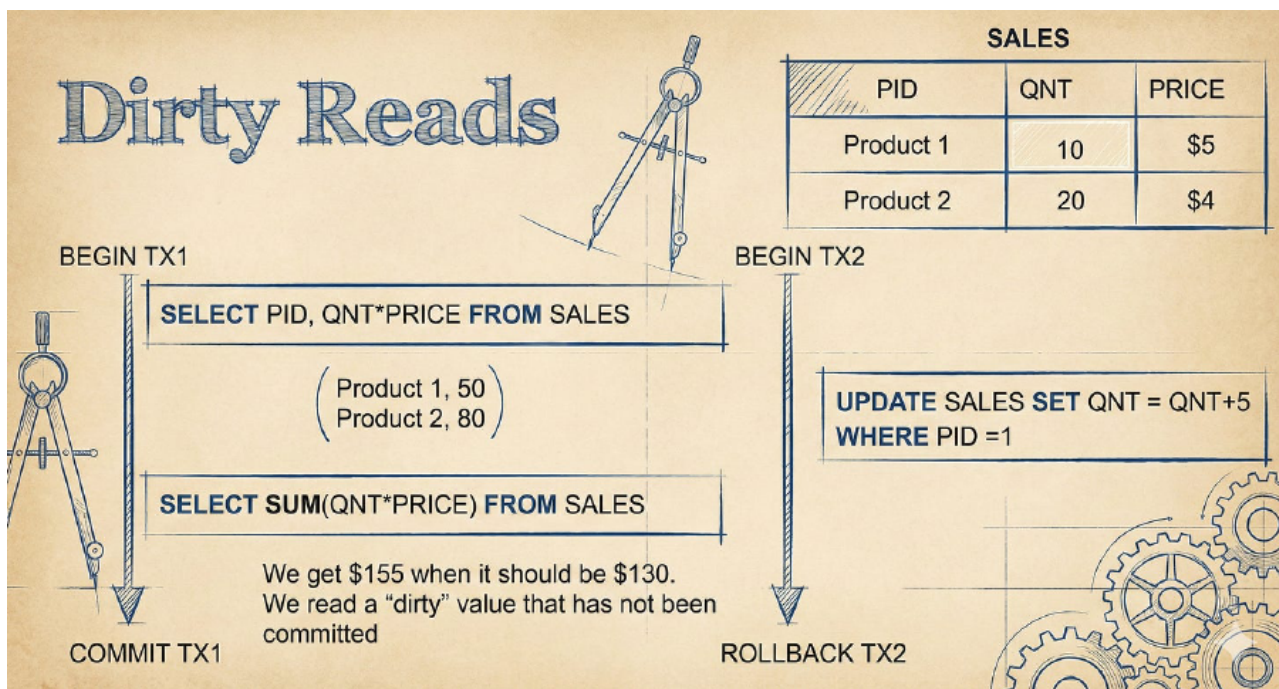
if two users are updating the same row and column at the same time, whose update should “win,” and when should the other user be allowed to see that new value?

Before we talk about isolation levels, we need to understand the concurrency anomalies they’re trying to prevent. These are the weird, unintuitive things that can happen when multiple transactions run at the same time.

Dirty Read

Transaction B reads data that Transaction A has modified but not yet committed.

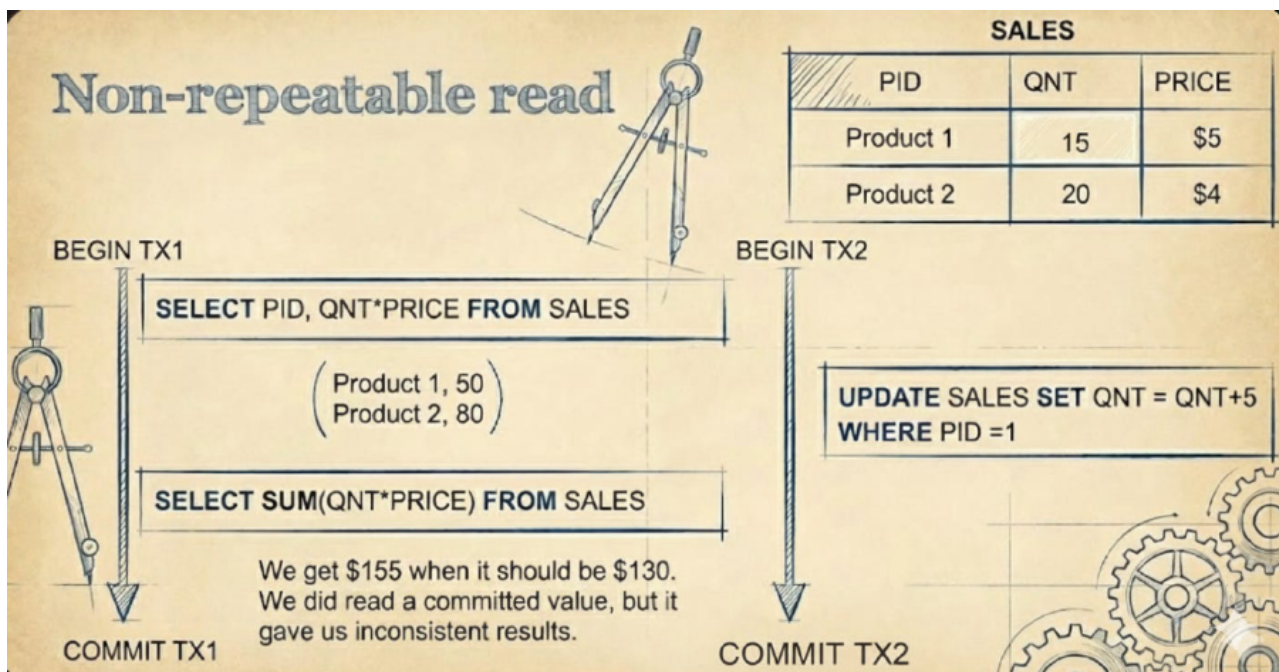
If Transaction A then rolls back, Transaction B ends up holding an invalid or “dirty” value that never truly existed in the committed state.



Non-repeatable Read

Within a single Transaction A, the same row is read more than once, while Transaction B modifies and commits that row in between those reads.

Transaction A then sees different values for the same row during its lifetime, so transaction A cannot "repeat" the same read and get the same result.



Phantom Read

Transaction A runs a query that returns a set of rows matching some condition.

Transaction B then inserts, updates, or deletes rows that satisfy that condition, so when Transaction A runs the same query again, it sees a different set of rows extra or missing "phantoms" within the same transaction.

NOTE: This is different from non repeatable reads because in non-repeatable reads, values in same row

changes, but in phantom reads, new rows are added or deleted.



Isolation level

Now that we've seen the anomalies (dirty reads, non-repeatable reads, lost updates, phantom reads), we can finally talk about isolation levels.

When multiple transactions run in parallel, the isolation level is basically the database's answer to:

"How much of Transaction B's work is Transaction A allowed to see (and when), and which anomalies are we willing to tolerate for the sake of performance?"

Repeatable Reads

Consistent reads within the same transaction meaning the idea is -> "If I read a row once in my transaction, I want to see the same value every time I read that row again."

So at Repeatable Read:

Dirty reads are prevented.

Non-repeatable reads are prevented (A's view of a given row is stable).

Phantom reads can occur

Now I start two transactions T1 and T2, concurrently. In the image below, let's say the left transaction is T1 and the right transaction is T2

This is how we check the isolation level (I've used MySQL; you can check the documentation for other databases.)

```
mysql> SELECT @@transaction_ISOLATION;
+-----+
| @@transaction_ISOLATION |
+-----+
| REPEATABLE-READ        |
+-----+
1 row in set (0.00 sec)
```

Here i update a row in T1 (and i havent committed yet), and if i read the same row from T2, T2 would still show the older value of the row (consistent read).

<pre>mysql> UPDATE users SET name = 'a_t1' WHERE id = 1; Query OK, 1 row affected (0.05 sec) Rows matched: 1 Changed: 1 Warnings: 0 mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a_t1 +----+-----+ 1 row in set (0.00 sec)</pre>	<pre>mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a +----+-----+ 1 row in set (0.05 sec) mysql></pre>
--	--

And now if i commit T1, and re-read the same row in T2 then again it would still read the same old value (haven't committed T2 yet). So this proves that Repeatable read is opaque to whatever has been done elsewhere in the system it still gives a stable consistent read value.

<pre>mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a_t1 +----+-----+ 1 row in set (0.00 sec) mysql> COMMIT; Query OK, 0 rows affected (0.04 sec) mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a_t1 +----+-----+ 1 row in set (0.00 sec)</pre>	<pre>mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a +----+-----+ 1 row in set (0.05 sec) mysql> SELECT * FROM users WHERE id = 1; +----+-----+ ID name +----+-----+ 1 a +----+-----+ 1 row in set (0.00 sec) mysql></pre>
---	--

Read Committed

T1 can only see changes from T2 after T2 has committed.

T1 will never read uncommitted data from T2.

So the idea here is -> "Only let me see committed data."

So at Read Committed:

No dirty reads (blocked)

But still:

Non-repeatable reads can happen

A reads a row -> B commits an update to that row -> A reads again -> sees a different value.

Phantom reads can happen

A queries a set of rows -> B inserts/deletes rows that match that condition -> A queries again and sees a different set.

Setting the isolation level for both transactions to READ COMMITTED.



The image shows two side-by-side terminal windows. Both windows show the command `mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;` being executed, with the response `Query OK, 0 rows affected (0.00 sec)`. Below this, both windows show the command `mysql> SELECT @@transaction_ISOLATION;` being executed. The output is a table with one row: `@@transaction_ISOLATION` with the value `READ-COMMITTED`. The response at the bottom of each window is `1 row in set (0.00 sec)`.

Read Uncommitted

Reads even uncommitted values from the transaction.

Idea -> "I don't care, just let me read whatever is there, even if it's not committed yet."

T1 is allowed to read changes that T2 has made but not committed yet.

If T2 rolls back, T1 has already used a value that never truly existed.

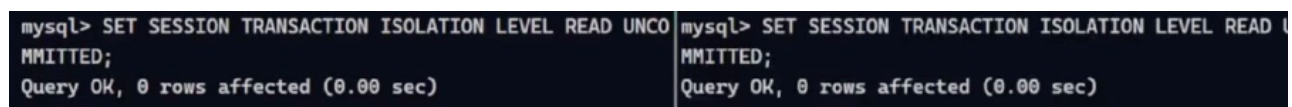
So at Read Uncommitted, the database allows:

Dirty reads

Non-repeatable reads

Phantom reads

Now again we will set the isolation level to READ UNCOMMITTED



The image shows two side-by-side terminal windows. Both windows show the command `mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;` being executed, with the response `Query OK, 0 rows affected (0.00 sec)`.

As you see we haven't committed yet in T2

Serializable

This is the strictest and the strongest isolation level because when one transaction reads, it blocks the others, so others will have to wait.

Every read is a locking read (it just blocks everything)

Idea -> "Pretend all transactions ran one after another, not in parallel."

So at Serializable:

Dirty reads – not allowed

Non-repeatable reads – not allowed

Phantom reads – not allowed

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

mysql> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)
```

As you can see this is what we mean by strict isolation level, as we haven't committed or rolled back T1 yet, T2 just waits there until a COMMIT or a ROLLBACK operation happens.

So to summarize this is how the isolation level and concurrency anomalies relate to:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Allowed	Allowed	Allowed
READ COMMITTED	Not Allowed	Allowed	Allowed
REPEATABLE READ	Not Allowed	Not Allowed	Allowed
SERIALIZABLE	Not Allowed	Not Allowed	Not Allowed

Now, coming to Durability

Durability

Once a transaction commits successfully, its changes must survive crashes, restarts, and power failures. The database is not allowed to "forget" a committed transaction.

How Databases Ensure Durability ?

Transaction Logs (Write-Ahead Logging) : Most relational databases rely on a Write-Ahead Log (WAL) to preserve changes before they're written to the main data files.

You can read more about WAL here:

Replication

Synchronous Replication Writes are sent to multiple nodes or data centers immediately. A transaction is only marked as committed when the primary and at least one replica (or a quorum, depending on the system) confirm that the data is safely stored. This reduces the risk of data loss if the primary fails right after commit.

Asynchronous Replication The primary node commits locally first and then ships changes to replicas in the background. This improves latency, but creates a small window where recent writes may be lost if the primary fails before the replicas have been updated.

Backups and Snapshots

Regular backups provide a safety net beyond logs and replication. In case of severe corruption, human error, or catastrophic failure:

Full Backups: Capture the entire database at a specific point in time.

Incremental/Differential Backups: Store only the changes since the last full (or previous) backup, making backups faster and more frequent.

Off-Site Storage: Keeps backups in a separate location so they survive localized disasters and hardware failures.

Hope you got a clear understanding of what ACID transactions are and how all of this works behind the scenes.

If you learnt something out of this then please do like and subscribe, this really would give me more motivation to create such in depth deep dives.

Take Care and See you in the next one. ??

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

How does the database guarantee reliability using write-ahead logging?

2025-12-03 · System Design from its first principles : Part 3

Source: <https://pradyumnachippigiri.substack.com/p/how-does-the-database-guarantee-reliability>

Appreciate you reading this newsletter, I hope the content and resources are helping you level up as an engineer. If you want me to write about something specific, just message me or comment on the post.

Note: This article on was originally published on my blog - <https://pradyumnachippigiri.dev/blogs/wal-logs>

In Part 1 and Part 2 we saw about ACID transactions and why they form the backbone for any Database.

For any database system, reliability is super important.

We keep saying “the transaction commits”... But what actually happens at commit? How does the database make that promise: “Your data is safe now”?

This is where the Write-Ahead Log (WAL) comes in.

1. What “Commit” Really Means ?

For any database system, reliability is non-negotiable.

When you run a query like:

you're not just asking the database to change a number. You're asking it to promise:

This change won't disappear if:

the power goes out,

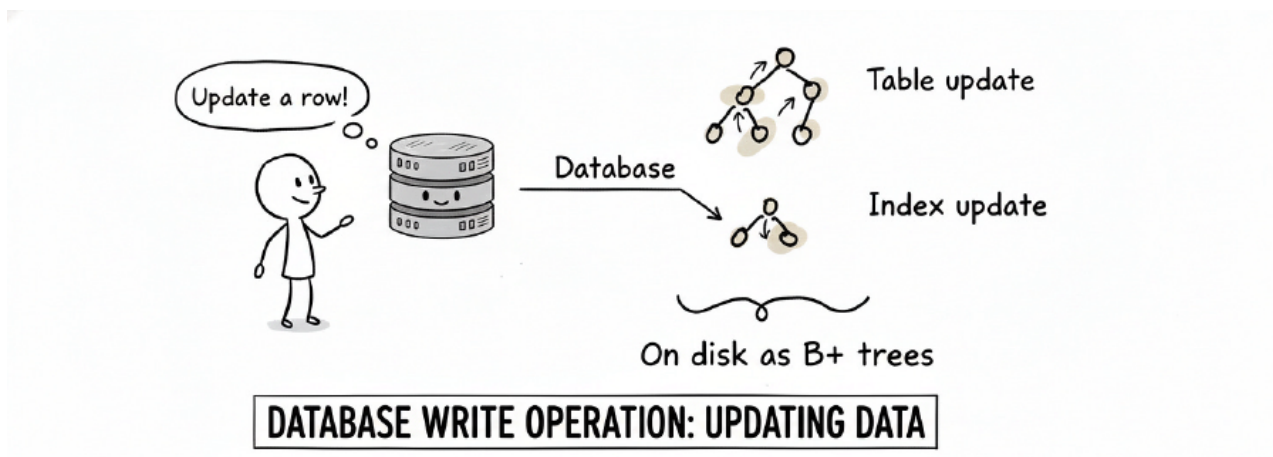
the OS crashes,

even certain kinds of hardware failure happen.

That promise only makes sense if the change is safely stored on non-volatile storage (disk/SSD/NVMe, etc.) something that doesn't lose data even when the machine restarts.

So a naive mental model of commit is:

“At commit, as soon as the row is updated and the transaction is committed, the corresponding blocks on the disk are changed both in the table and in any indexes.”



That's kinda the right intuition... but it hides a ton of engineering complexity.

Because if we literally updated the data files and all indexes on every commit, we'd be:

Doing many random writes all over disk.

Taking a big performance hit.

Still vulnerable to partial writes in the middle of a crash.

So how do databases stay fast and safe?

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

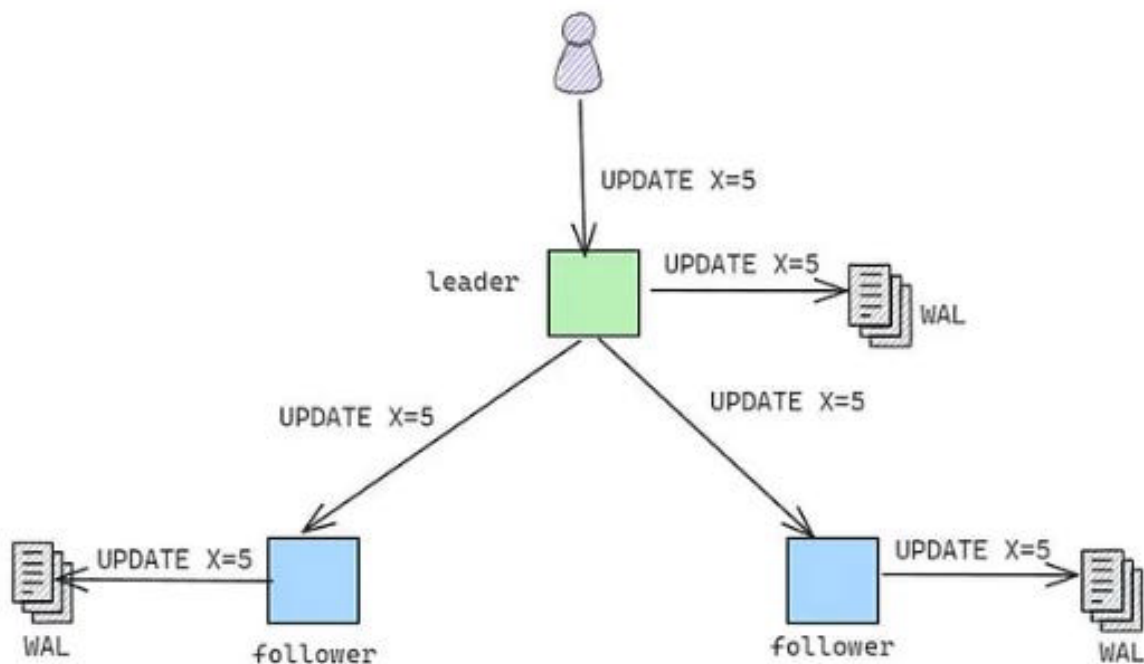
2. Enter WAL: Log First, Data Later

The standard trick is the Write-Ahead Log (WAL).

WAL is a standard method for ensuring data integrity. It's not just used in traditional RDBMS the idea can be used anywhere we care about reliability and crash recovery.

The core idea is beautifully simple:

Before making changes to the actual database files, log those changes in a separate, append-only log file.



Each node in a database system keeps an append-only log file in persistent storage and every client update sent to the node is first appended to this file (write-ahead) and later applied to the data in the database. This logging technique is called Write-Ahead Logging because the update commands are written to the logs before the actual updates on the data are performed.

Think of them like this -

The Database: The official, heavy ledger book where you neatly write final balances and long-term records. The WAL: A scratchpad where you quickly jot down every transaction as it happens, in order, before you calmly copy them into the ledger later.

Why is this powerful?

The WAL is a sequential, append-only file. Appending to the end is much cheaper than jumping around updating different parts of tables and indexes.

Updating the main data files can be done later, in the background.

As long as the change is safely written to the WAL, the database can say:

“Commit successful and your data is durable.”

So instead of:

“Every commit = update all relevant data/index blocks on disk”

so the entire process would look somewhat like this :

Log the change to WAL (sequential write).

Flush WAL to disk (ensure it's persisted).

Writing to the WAL file is not the same as having it safely on disk.

There are layers:

Your DB writes to its own WAL buffer in RAM.

OS writes that to the page cache in RAM.

Only when the OS actually issues a disk write + flush do the bytes land on physical storage.

If the power goes out while the bytes are only in RAM (DB buffer or OS cache) -> they're gone.

So on COMMIT, the DB must ensure:

The WAL bytes for this transaction are forced to disk.

This is often done with something like `fsync()` (or equivalent), which tells the OS:

"No seriously, push this to stable storage now."

Key idea: A transaction is only considered durable when the WAL records for it have definitely hit disk.

Mark transaction as committed.

Later, apply those changes to the actual data files during a background process (checkpoint).

3. Advantages of WAL

Fewer disk writes -> better performance (Updating table data and indexes directly can mean a lot of random I/O)

Point-in-time recovery (PITR) (Incase of failure, can restore a backup of the database.)

4. But what if the crash happens while writing the WAL?

Good question and it's not just theoretical. Imagine we're appending a record to the WAL:

The record is, say, 100 bytes long.

The system crashes after 43 bytes are written.

Now we have a truncated / half-written log record. How does the database know:

whether this record is valid or corrupted?

whether it should replay it during recovery?

This is where data protection inside WAL itself comes in.

5. CRC-32: Guarding each log record

Every serious implementation of WAL protects each record with a checksum. A common choice is CRC-32 (Cyclic Redundancy Check).

Rough idea:

When the WAL record is created:

The system calculates a CRC-32 checksum of its contents.

That checksum is stored alongside the record.

During:

Crash recovery, or

Replication (streaming WAL to another server)

The database:

Recomputes the CRC-32 of the record.

Compares it with the stored checksum.

If the checksums don't match?

The record is considered corrupted / incomplete.

6. Under the Hood: Segments, Pages, and LSNs

WAL itself isn't just a single infinitely growing file. It's usually a set of files on disk.

You'll often see terminology like:

WAL segments Each file in the WAL is called a segment. In systems like PostgreSQL, each segment is roughly 16 MB.

Pages inside segments Each segment is split into smaller pages often 8 KB each. The database writes WAL records onto these pages as it appends data.

This lets the database:

Rotate old segments,

Archive them for backups,

Delete them once they're no longer needed (e.g., after a backup + checkpoint).

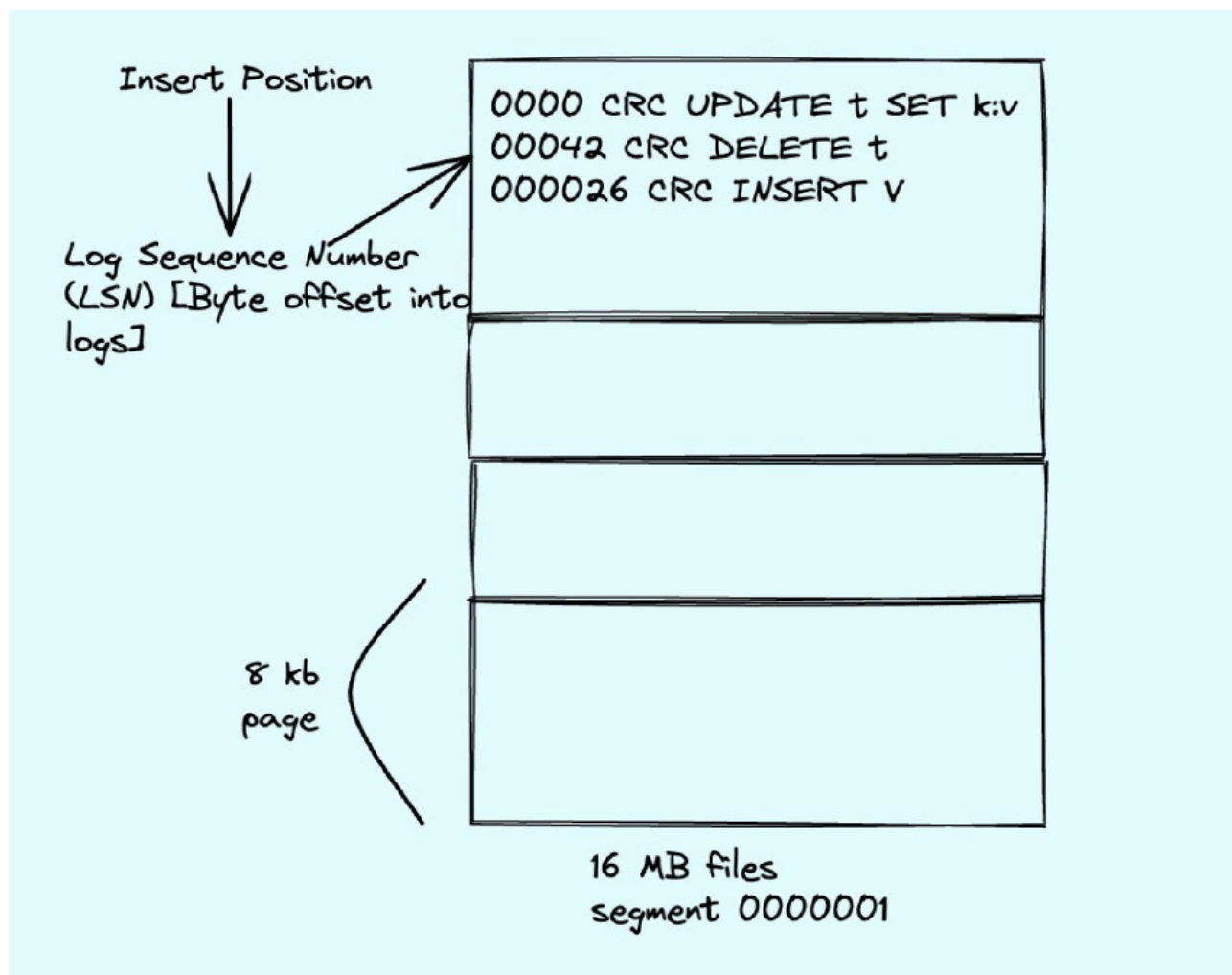
Log Sequence Number (LSN)

Every WAL entry is given an identifier called the Log Sequence Number (LSN).

It's not simply 0, 1, 2, 3,

Conceptually, it's like a byte offset into the WAL stream:

"At this exact byte position in the log, this record starts"



LSNs are used to:

Track how far the database has applied WAL to the main data files.

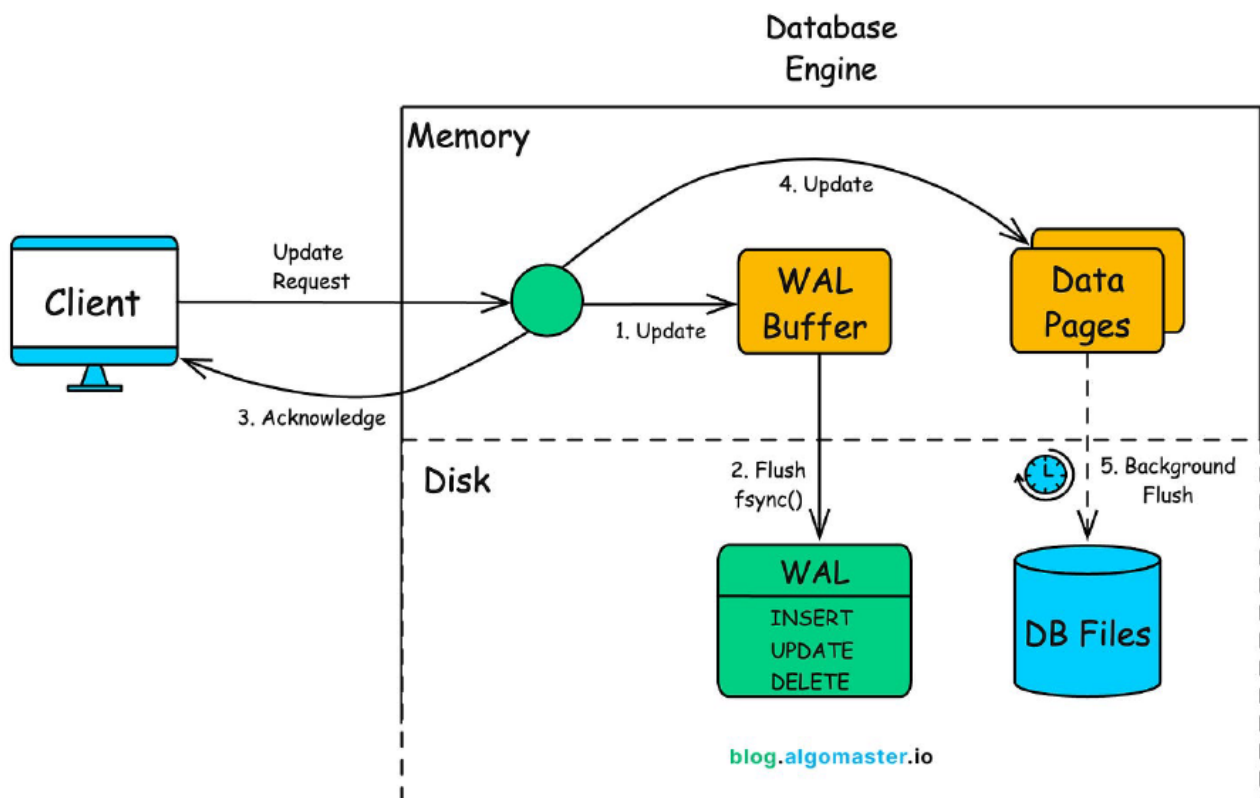
Coordinate replication (e.g., a standby server has replayed up to LSN X).

Implement point-in-time recovery (replay up to a certain LSN and then stop).

So you can think of the WAL as a big ordered timeline of changes, and the LSN is your precise timestamp on that timeline.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

7. How This All Ties Back to Commit



Phase 1: The Active Transaction

Request: You send an UPDATE or INSERT command.

Modify Memory (RAM): The database locates the specific data page in its Buffer Pool (RAM) and modifies it right there.

Status: This page is now marked as a "Dirty Page" because the version in RAM is newer than the version on the hard drive.

Generate Log: Simultaneously, the database creates a WAL record describing this change (including the LSN - Log Sequence Number).

Phase 2: The Commit (Durability)

Request: You call COMMIT.

Prepare WAL: The database calculates the CRC-32 (checksum) for the log records.

Flush WAL to Disk: It forces only the WAL records (up to the current LSN) to be written physically to the disk.

Note: The main data file on the disk is still old. The new data exists only in the WAL (on disk) and the Dirty Page (in RAM).

Acknowledge: The database tells you: "Commit successful."

Phase 3: The Background Checkpoint (The "Dirty Page" Flush)

This happens asynchronously, minutes later, or when the log gets too big.

Identify: The Checkpoint process looks at the Buffer Pool for "Dirty Pages" (pages modified in RAM but not yet on disk).

Flush Data: It writes these Dirty Pages from RAM directly to the Main Table/Index Files on the disk.

Sync: It ensures these data file writes are physically synced to the hardware.

Truncate Log: It marks the WAL segments prior to this checkpoint as "recyclable" or "ready for archive," because that data is now safely inside the main data files.

Phase 4: Crash Recovery (If the plug is pulled)

Scenario: The power failed after Phase 2 (Commit) but before Phase 3 (Checkpoint).

Restart: The database engine boots up.

Locate Checkpoint: It reads and finds the LSN of the last successful checkpoint.

Logic: "Everything before this LSN is guaranteed to be in the data files. I can ignore it."

Scan Forward: It reads the WAL files starting from that Checkpoint LSN up to the end of the log.

Verify: It checks the CRC-32 of the log entries to ensure the crash didn't corrupt the last half-written log line.

Redo (The Replay):

It sees a committed transaction in the WAL.

It checks the data page. The page has an older LSN (because the RAM was wiped out).

It applies the change from the WAL to the page to restore the data.

Undo (The Rollback):

It sees a transaction in the WAL that started but has no "COMMIT" record.

It discards those changes to ensure consistency.

I know this article felt like too much words, and less visuals. But the topic is such, i had to cover everything that happens behind the scenes. But hope you got a fair understanding of WAL and how databases manage the heavy lifting for us behind the scenes. If you want more such powerful and extremely informative content right to your inbox, then please do follow and subscribe.

Thanks for reading LevelYouUp! Subscribe for free to receive new posts and support my work.

Thanks for reading LevelYouUp! This post is public so feel free to share it.

Share